

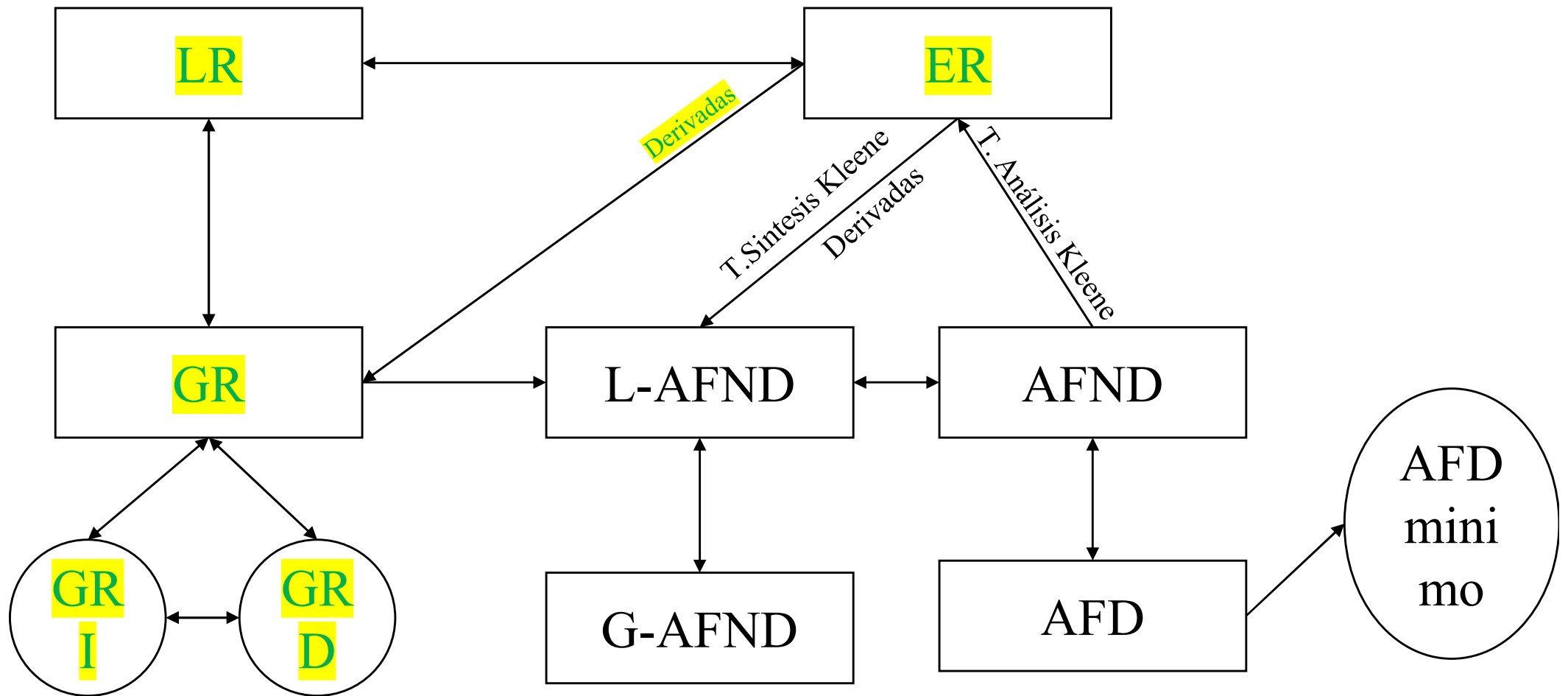
Tema 4

Autómatas Finitos



Esquema general de la asignatura.

Tema 4.



TEMA 4. Lenguajes Regulares y Autómatas Finitos

- Autómatas finitos deterministas.
- Autómatas Finitos no deterministas.
- Teoremas de kleene
- La clase de los lenguajes regulares
- Problemas de decisión de los lenguajes regulares



Autómatas Finitos

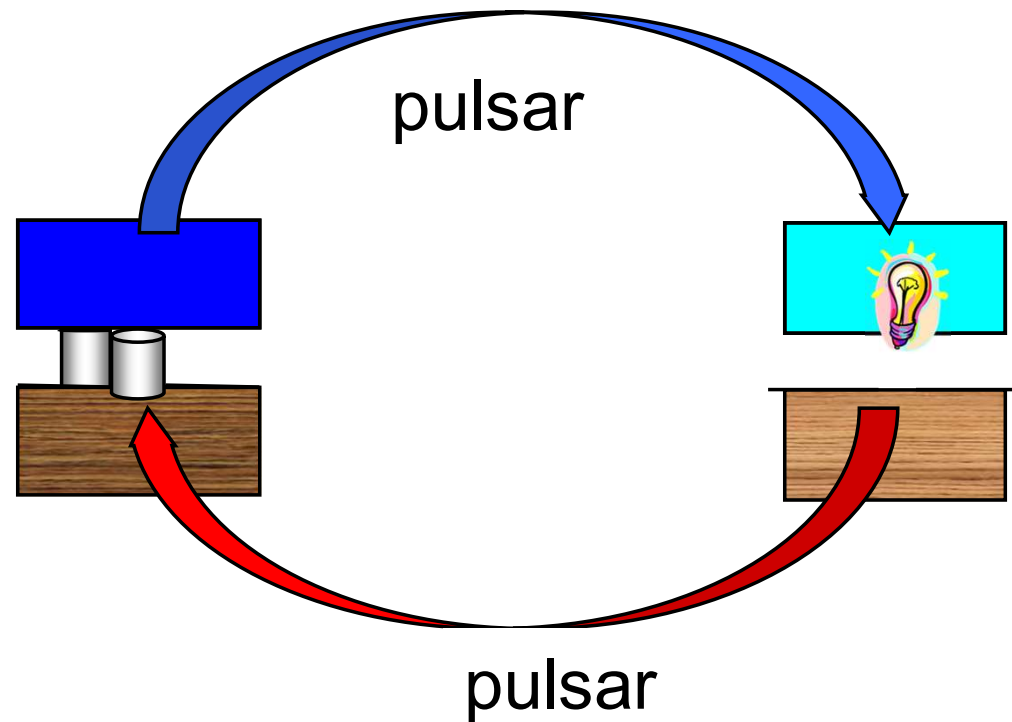
Introducción. Ejemplo sencillo: **un interruptor**

- Resulta claro que el interruptor puede encontrarse en dos situaciones distintas: **apagado** o **encendido**.
- El interruptor sólo puede recibir un estímulo exterior: **que se pulse su botón**.
- Cuando el interruptor esté apagado, si se pulsa el botón, pasa a estar encendido, en otro caso sigue apagado.
- Cuando el interruptor esté encendido, si se pulsa el botón, pasa a estar apagado, en otro caso sigue encendido.
- Es frecuente que inicialmente el interruptor esté apagado.
- En este caso, puede considerarse que la salida del sistema es el funcionamiento del dispositivo que controle (una bombilla, por ejemplo).



Autómatas Finitos

La siguiente figura muestra gráficamente la situación.



Autómatas Finitos

El análisis del comportamiento del autómata del interruptor sugiere dividir la descripción formal del mismo en dos partes:

- **Entradas** (externas),
 - Que representan los estímulos del exterior a los que puede reaccionar el autómata.
- **Estados** (internos) y **transiciones**,
 - Los **estados** representan las diferentes situaciones en las que se puede encontrar el autómata. En el caso del **interruptor**: encendido y apagado
 - Las **Transiciones** representan el cambio del autómata de un estado a otro debido a las entradas.



Autómatas Finitos

- Definición 2.5: Se llama **autómata finito determinista** o simplemente autómata finito, y lo denotaremos por AF (o AFD), a la quintupla:

$$(\Sigma, Q, \delta, q_0, F)$$

donde:

- 1) Σ es un alfabeto llamada alfabeto de entrada
- 2) Q es un conjunto finito y no vacío de estados
- 3) δ es una función $\delta: Q \times \Sigma \rightarrow Q$ llamada función de transición
- 4) q_0 es un elemento de Q llamado estado inicial
- 5) $F \subseteq Q$ llamado conjunto de estados finales o de aceptación



Autómatas Finitos

El interruptor formalizado:

$$A_I = (Q_I, \Sigma_I, \delta_I, \theta, \Phi)$$

donde

- El **conjunto de estados** representa **encendido** con 1 y **apagado** con 0 .

$$Q_I = \{0, 1\}$$

- El **alfabeto de entrada** representa **pulsar** con p .

$$\Sigma_I = \{p\}$$

- El **estado inicial** q_0 es que esté **apagado** (0).
- En este autómata no se indican **estados finales** (Φ es el conjunto vacío).
- La **función de transición** se describe más adelante...



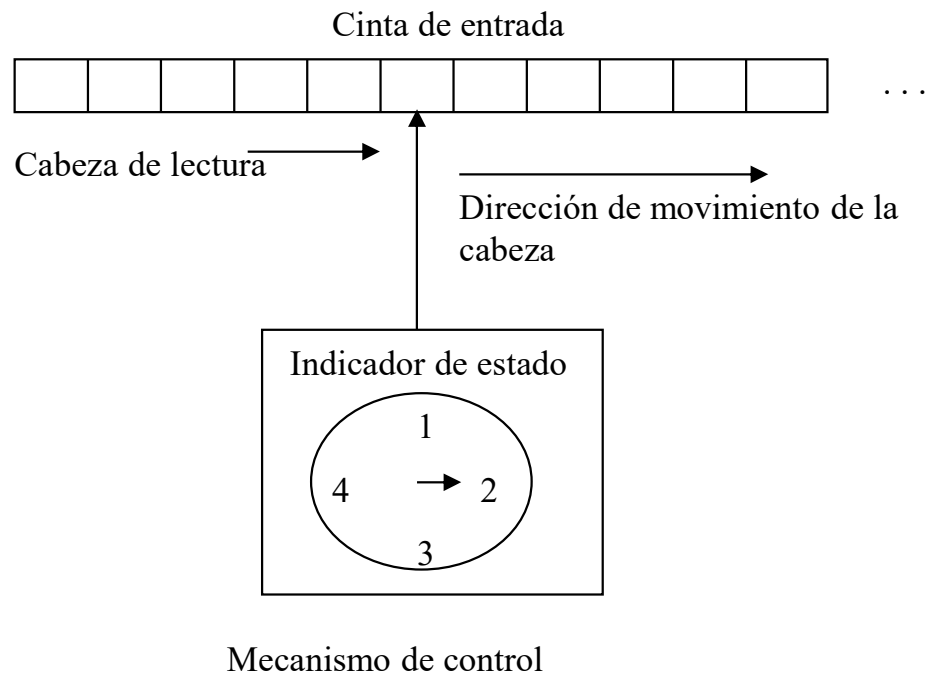
Autómatas Finitos

- La *función de transición* δ es una función $\rightarrow$
 $\rightarrow$ Para cada par $(q, a) \in Q \times \Sigma$ existe un y sólo un $p \in Q$ tal que $\delta(q, a) = p$
- Es por la existencia de esta función por lo que estos mecanismos se denominan “deterministas”.



Autómatas Finitos

- Otra visión de un autómata:
 - una cinta de entrada
 - un dispositivo señalador
 - un dispositivo “indicador de estado”
 - un mecanismo de control (programa de la máquina), evalúa δ



Autómatas Finitos

- Definición 2.6: Un par $(q, w) \in Q \times \Sigma^*$ es una **configuración** para $M = (\Sigma, Q, \delta, q_0, F)$ y expresa que la máquina se encuentra en el estado q cuando a la derecha del señalador se encuentra la palabra w .
- Definición 2.7: Definimos como un **paso de cálculo** o de cómputo de un autómata finito al paso de una configuración a otra y lo denotaremos por
$$(q, w) \mid \text{---}_M (p, v) \Leftrightarrow \exists x \in \Sigma / w = xv \wedge \delta(q, x) = p$$



Autómatas Finitos

- Definición 2.8: Sea $M = (\Sigma, Q, \delta, q_0, F)$ un AF, definimos el lenguaje aceptado por M , $L(M)$ como
$$L(M) = \{w \in \Sigma^* / \exists p \in F, (q_0, w) \vdash_M \dots \vdash_M (p, \lambda)\}$$
$$= \{w \in \Sigma^* / M \text{ acepta a } w\}$$
- Observaciones:
 - 1) $L(M)$ está formado por **todas las cadenas aceptadas por M** , y no un conjunto de cadenas que son todas aceptadas por M
 - 2) Si $F = \emptyset$, $L(M) = \emptyset$ y si $F = Q$, $L(M) = \Sigma^*$



Representación de AF's

a) Representación de AF en forma de tabla

1) Representamos δ en una tabla:

	símbolos de Σ
estados	$\delta(q_i, x)$

2) Marcamos con una $\rightarrow$ el estado inicial

3) Marcamos con asteriscos (*) los estados de F (estados finales)



Representación de AF's

b) Diagrama de transiciones

- 1) Construimos un diagrama de transiciones completo donde:
 - cada estado se representa como un nodo
 - Si $\delta(q_i, x) = q_j$, se dibuja un arco desde q_i a q_j y se etiqueta con “x”
- 2) Marcamos con una flecha $\rightarrow$ el nodo que representa en estado inicial
- 3) Marcamos con un doble círculo los nodos que representan estados finales de autómatas



Autómatas finitos

Ejemplo

- Se desea diseñar un autómata finito determinista que reconozca la palabra *pepe*:
- El alfabeto de entrada contendrá el siguiente conjunto (podría extenderse a más letras)

$$\{p, e\}$$

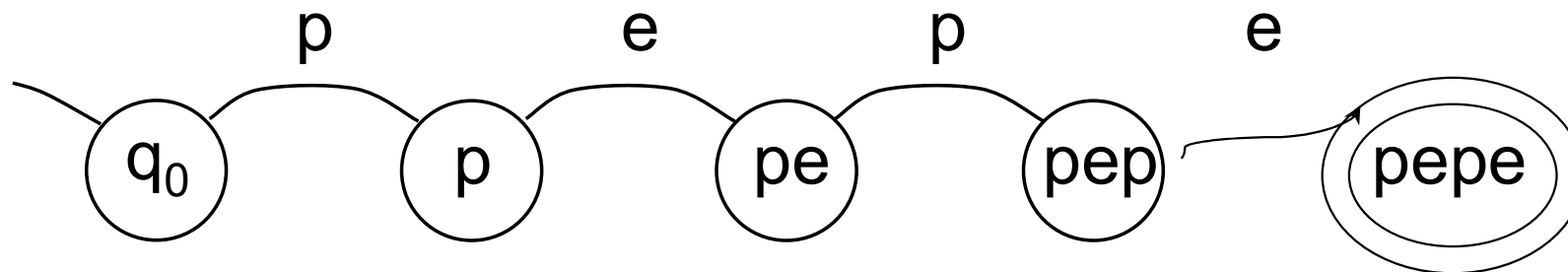
- Se puede analizar su funcionamiento determinando la parte relevante de su historia que tiene que corresponder a estados:
 - El autómata necesita un estado inicial (q_0)
 - El único cambio relevante en el estado inicial es que se reciba una letra p , en este caso debe transitarse a un estado que lleve cuenta de que de la palabra pepe sólo se ha recibido la primera letra (q_p o simplemente p)
 - En este estado el único cambio relevante es la presencia en la entrada de la letra e ; con ella debe transitarse a un estado que lleve cuenta de que se han recibido las dos primeras letras de la palabra (pe)
 - Este mismo razonamiento se aplica para añadir los estados correspondientes a pep y a $pepe$.
 - Cuando se llega a este último estado, puede afirmarse que el autómata ha reconocido la palabra completa por lo que será final.



Autómatas finitos

Diagrama de transición del ejemplo

- A continuación se muestra parte del diagrama de la función de transición del ejemplo.



Esta primera parte del ejemplo permite extraer una “técnica de diseño generalizable”

NOTA: ¿qué le falta al gráfico para ser un AFD?



Autómatas finitos

Tabla de transición del ejemplo

A continuación se muestra la tabla correspondiente a la situación estudiada.

	p	e
$\rightarrow q_0$	p	
p		pe
pe	pep	
pep		pepe
*pepe		



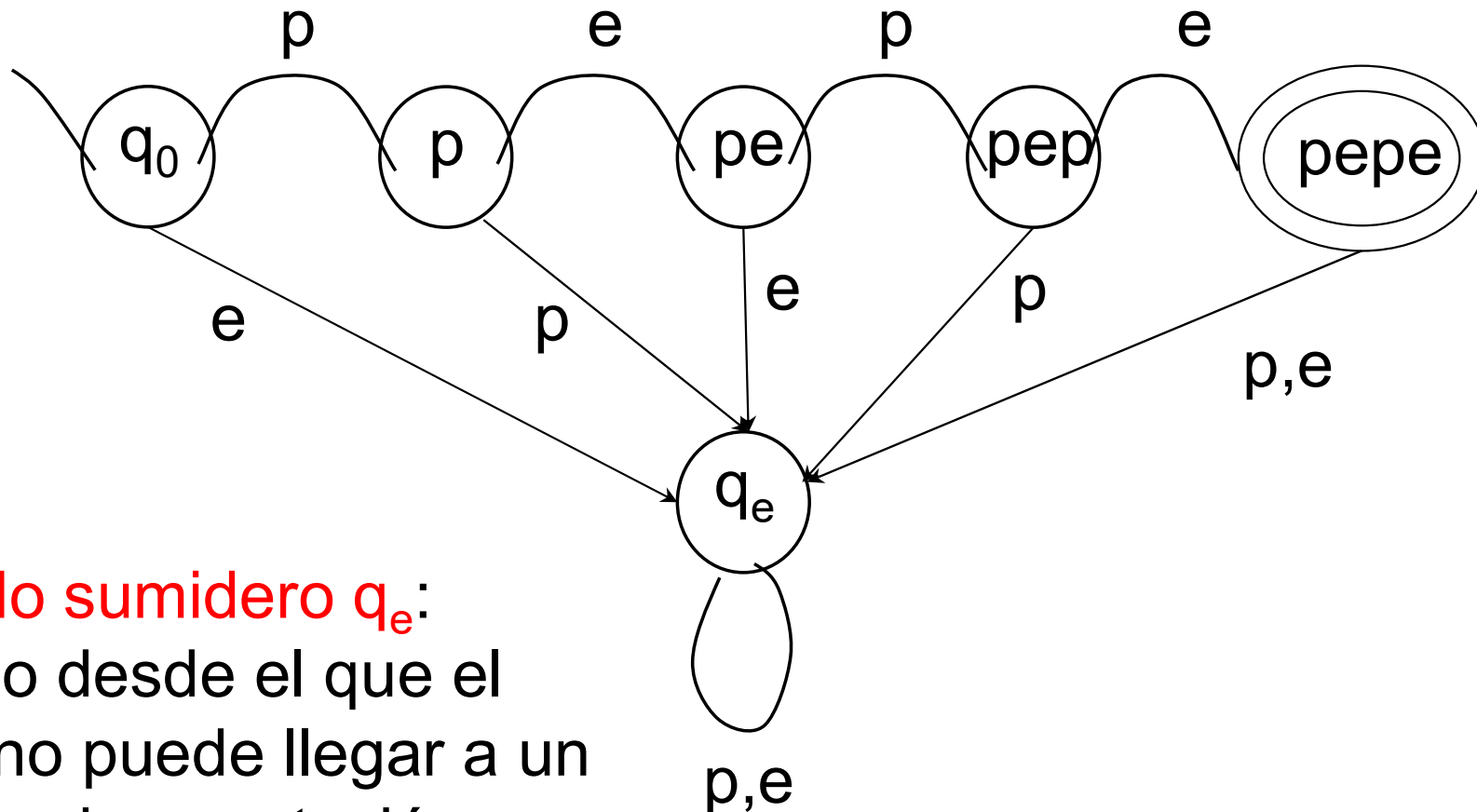
Autómatas Finitos

Ejemplo (cont.)

- Esto no es suficiente para definir por completo el autómata. La función de transición debe especificar estados para todas las combinaciones posibles de estados y símbolos de entrada. La tabla mostraba 6 casillas vacías:
 - Cuando el autómata se encuentra en el estado inicial, si recibe la letra e , puede afirmar que no va a poder reconocer la palabra *pepe* porque empieza por una letra (p) distinta a la recibida.
 - Lo mismo pasa cuando se está en el estado p y se recibe otra p , no se podrá terminar con éxito ya que *pepe* no comienza por pp .
 - Y la situación es la misma para las casillas (pe,e) y (pep,p) .
 - Desde el estado final, ya que el autómata sólo está interesado en reconocer la palabra *pepe*, y no las palabras que comiencen con *pepe*, cualquier otro símbolo que la siga debe implicar que el autómata no termina con éxito.
- Se puede añadir un estado **de error** (q_e) que recoja los descritos anteriormente:
 - Este estado recoge las transiciones no exitosas de los demás.
 - También es necesario especificar qué pasa cuando desde este estado, se reciben símbolos de entrada: resulta claro que, producida una situación de error, no hay ningún símbolo que pueda hacer que el autómata termine con éxito.



Diagrama de transición del ejemplo (completo)



Estado sumidero q_e :
estado desde el que el
AFD no puede llegar a un
estado de aceptación



Autómatas Finitos

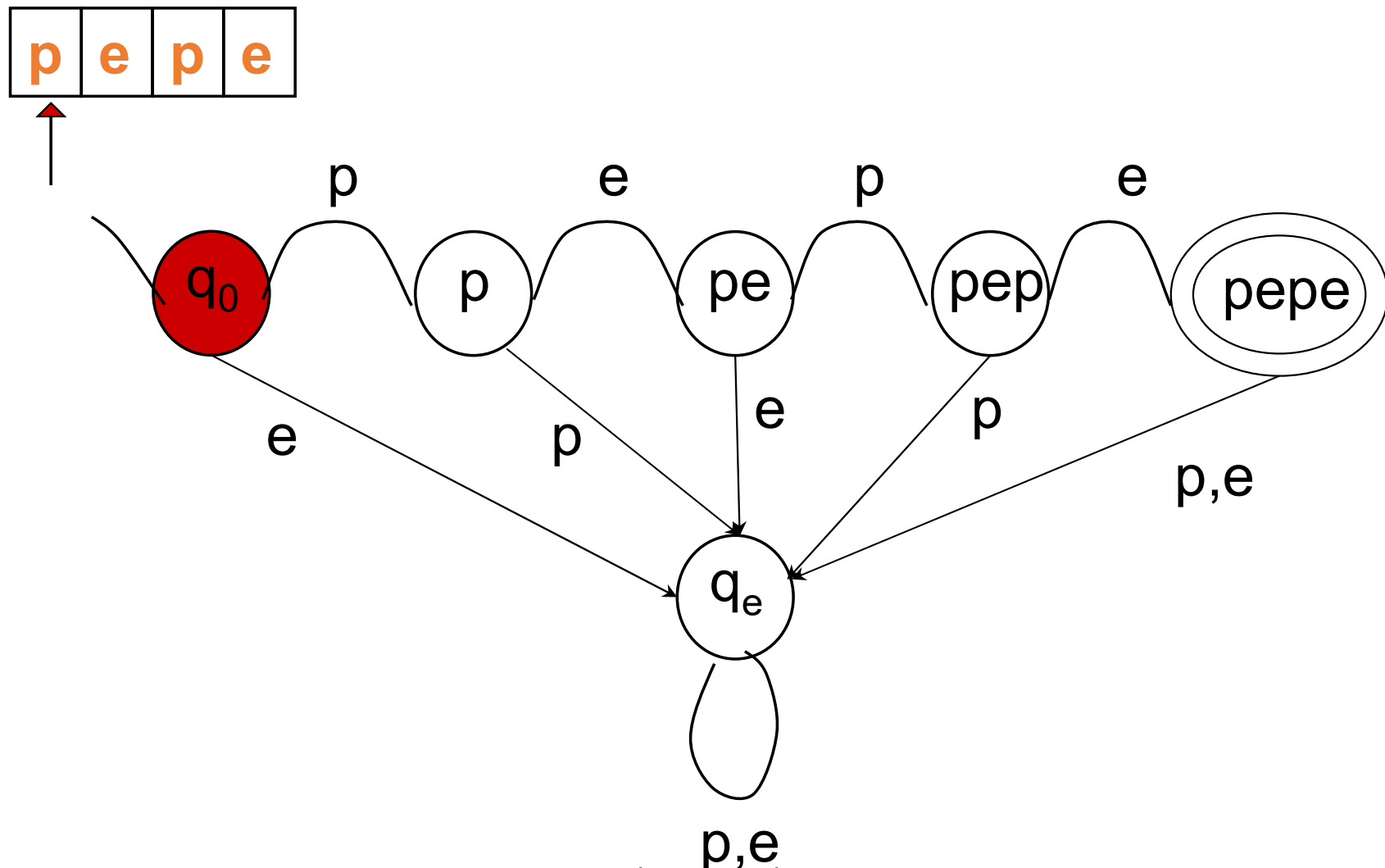
Tabla de transición del ejemplo (tabla completa)

	p	e
$\rightarrow q_0$	p	q_e
p	q_e	pe
pe	pep	q_e
pep	q_e	pepe
*pepe	q_e	q_e
q_e	q_e	q_e



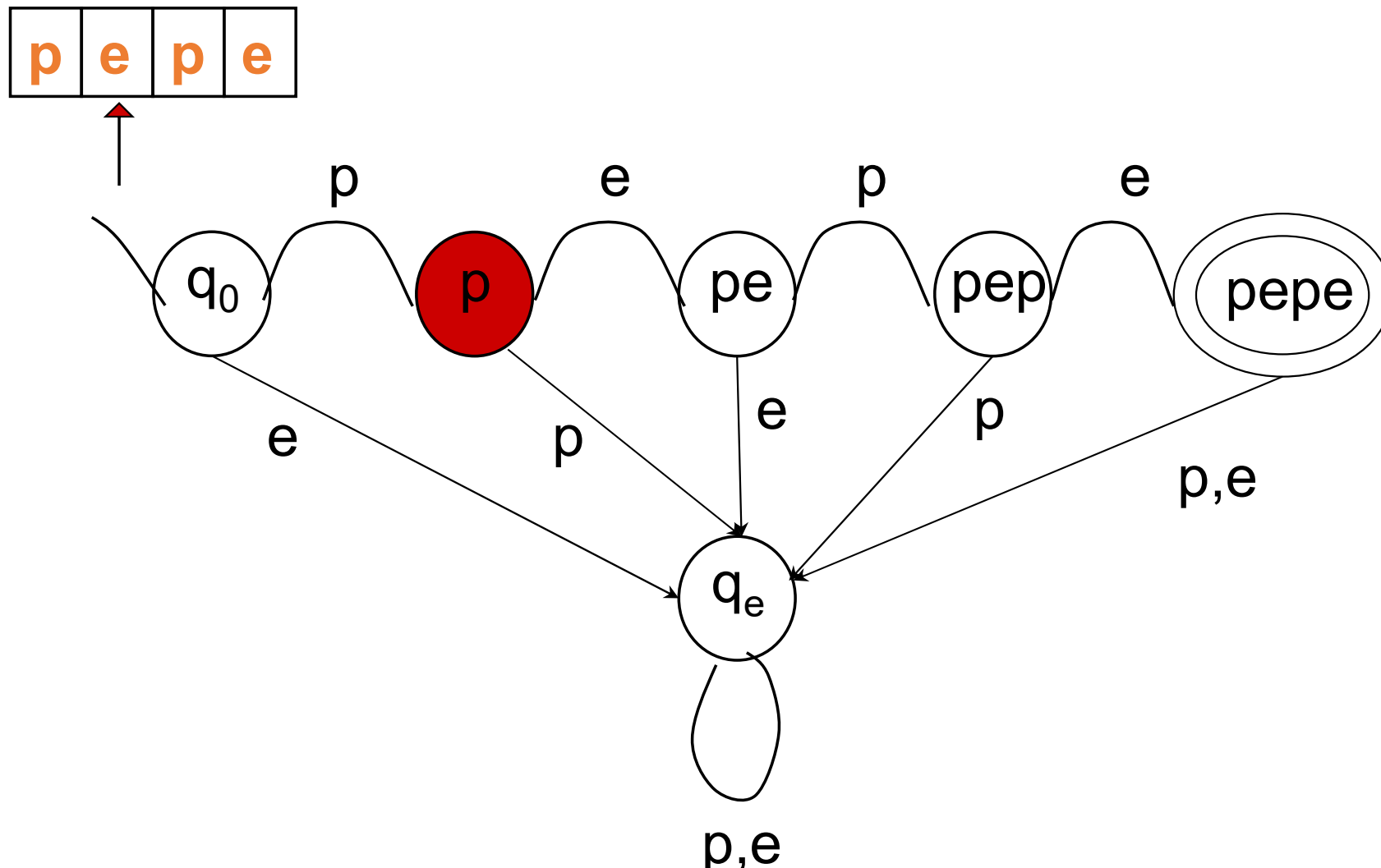
Autómatas finitos

Ejemplo: tratamiento de palabras. Procesamiento de entradas



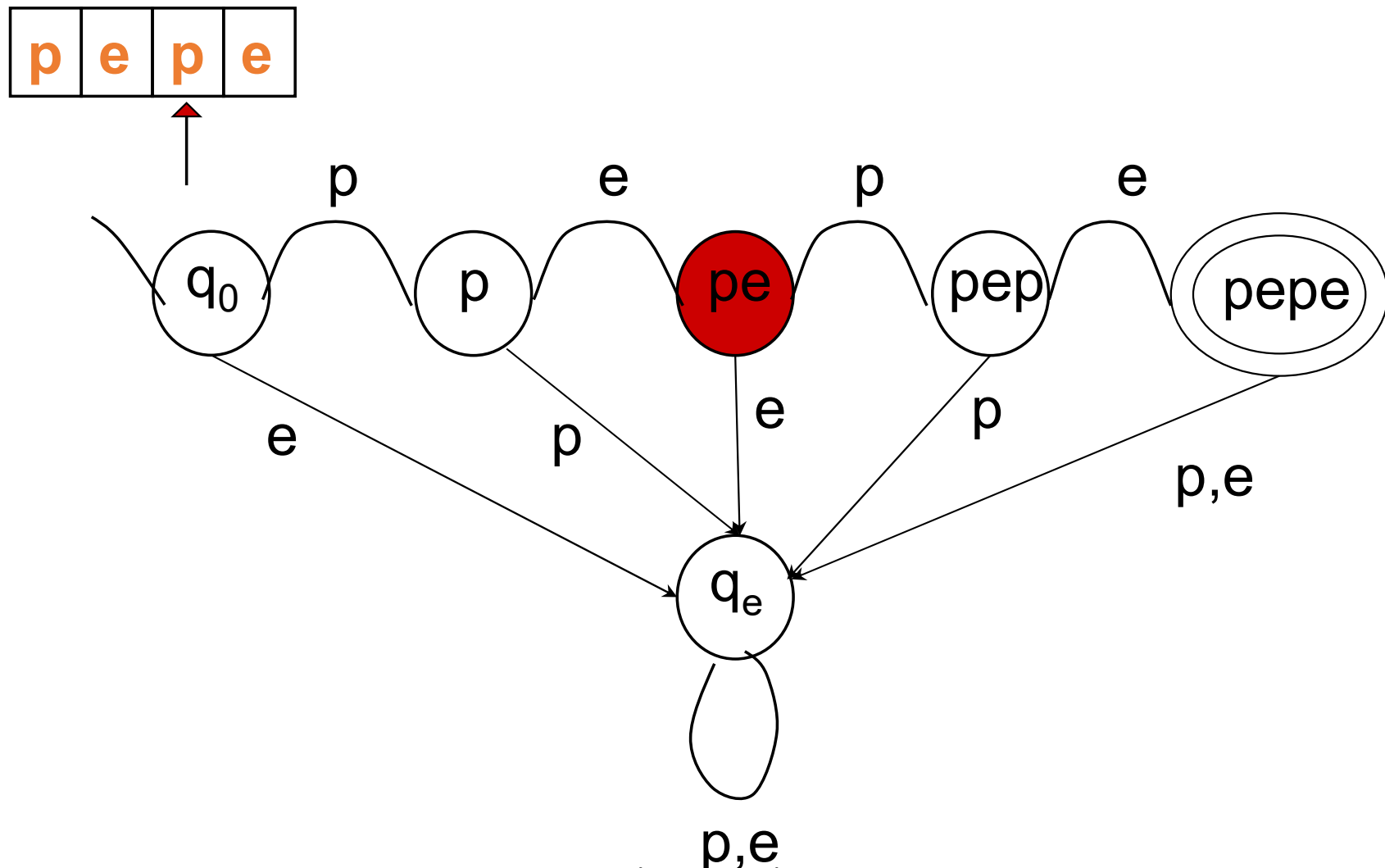
Autómatas finitos

Ejemplo: tratamiento de palabras. Procesamiento de entradas



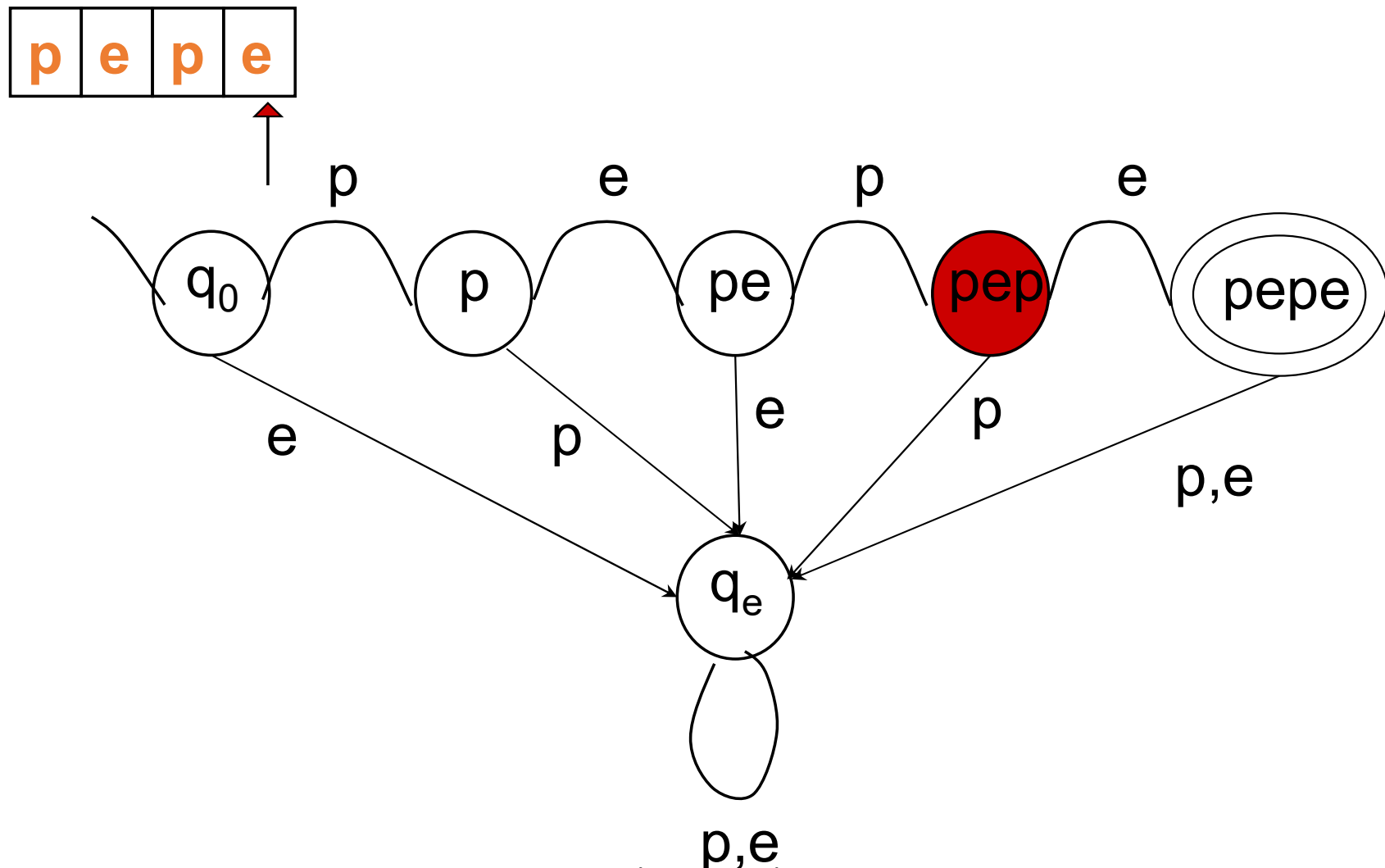
Autómatas finitos

Ejemplo: tratamiento de palabras. Procesamiento de entradas



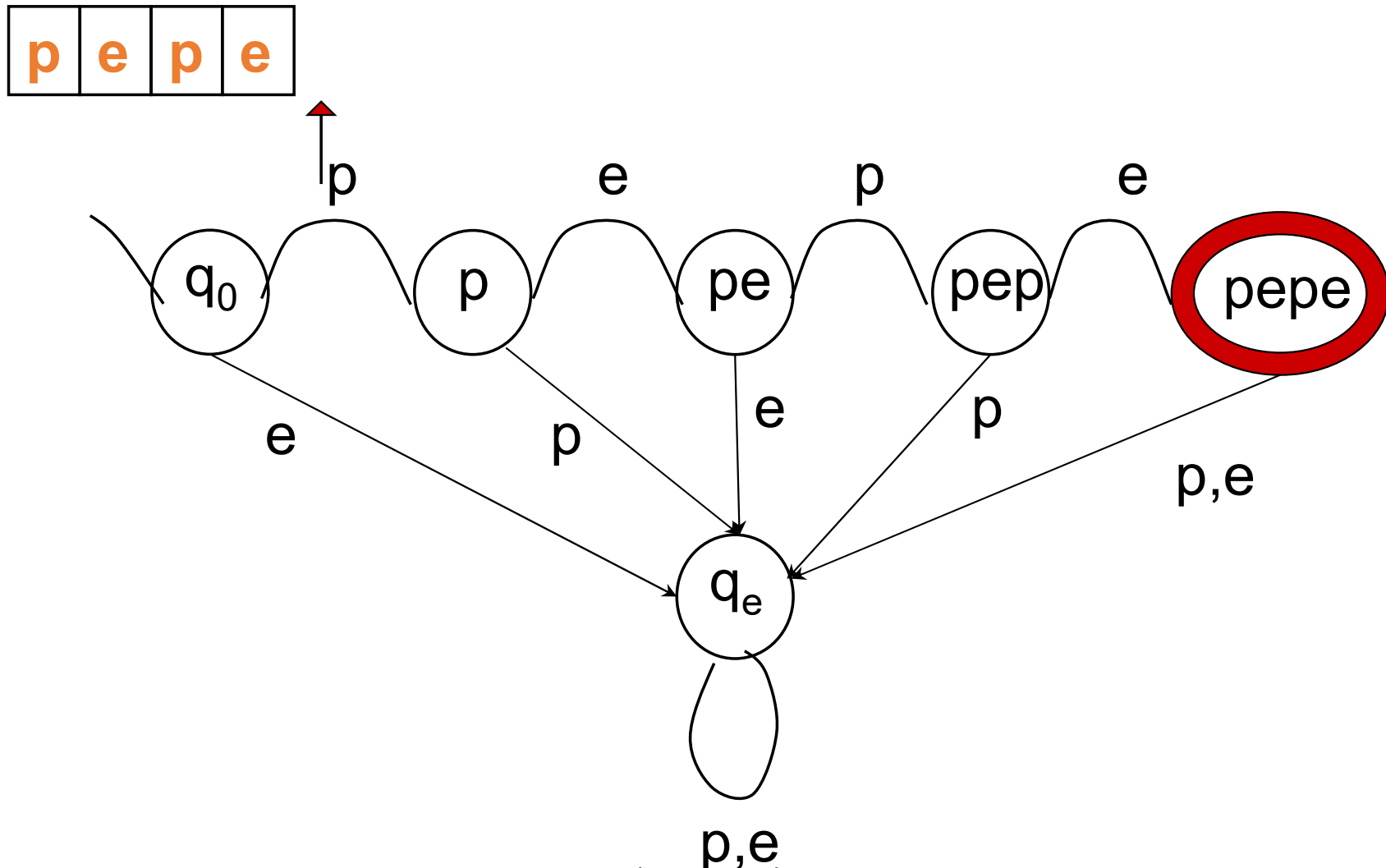
Autómatas finitos

Ejemplo: tratamiento de palabras. Procesamiento de entradas



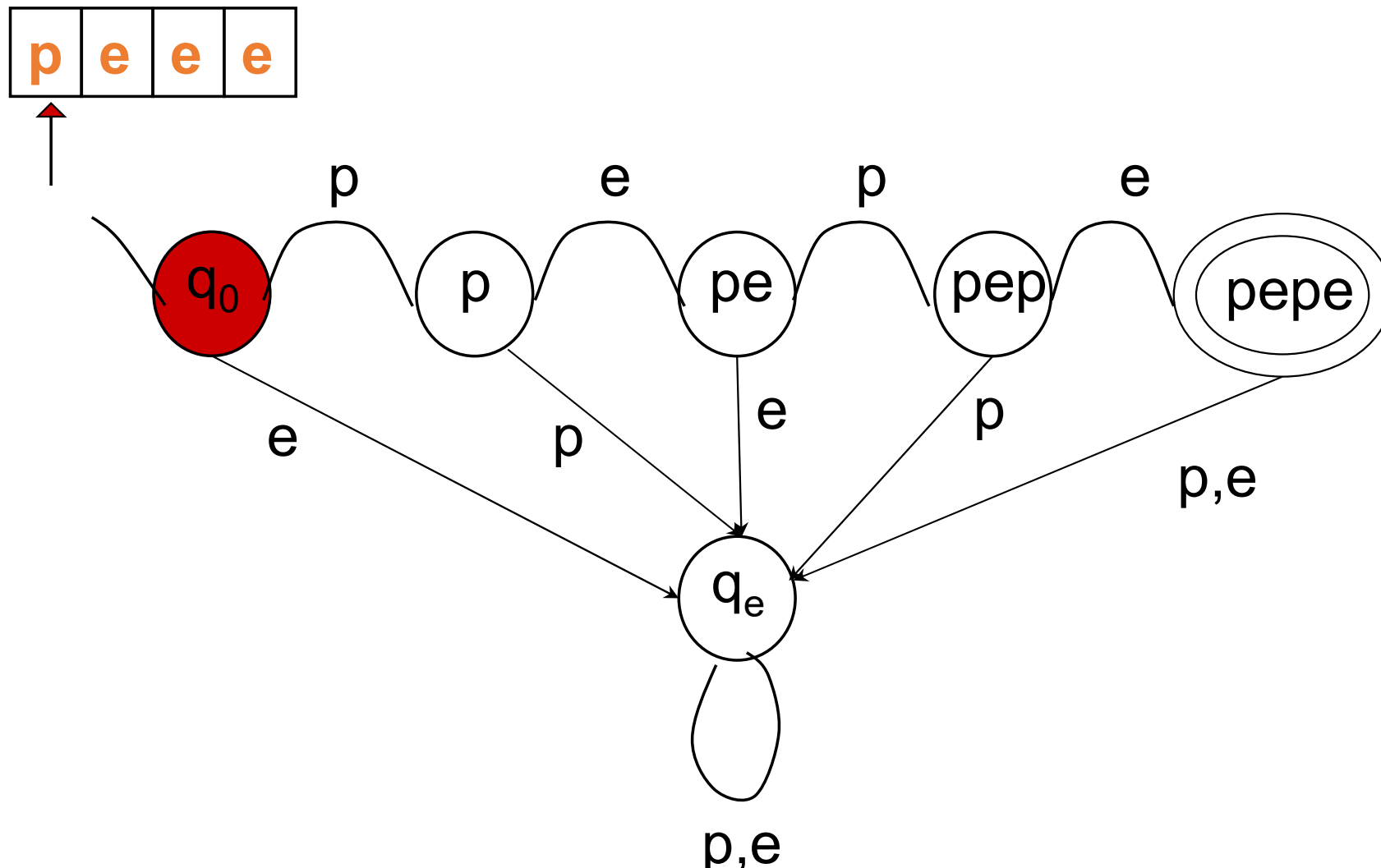
Autómatas finitos

Ejemplo: tratamiento de palabras. Procesamiento de entradas



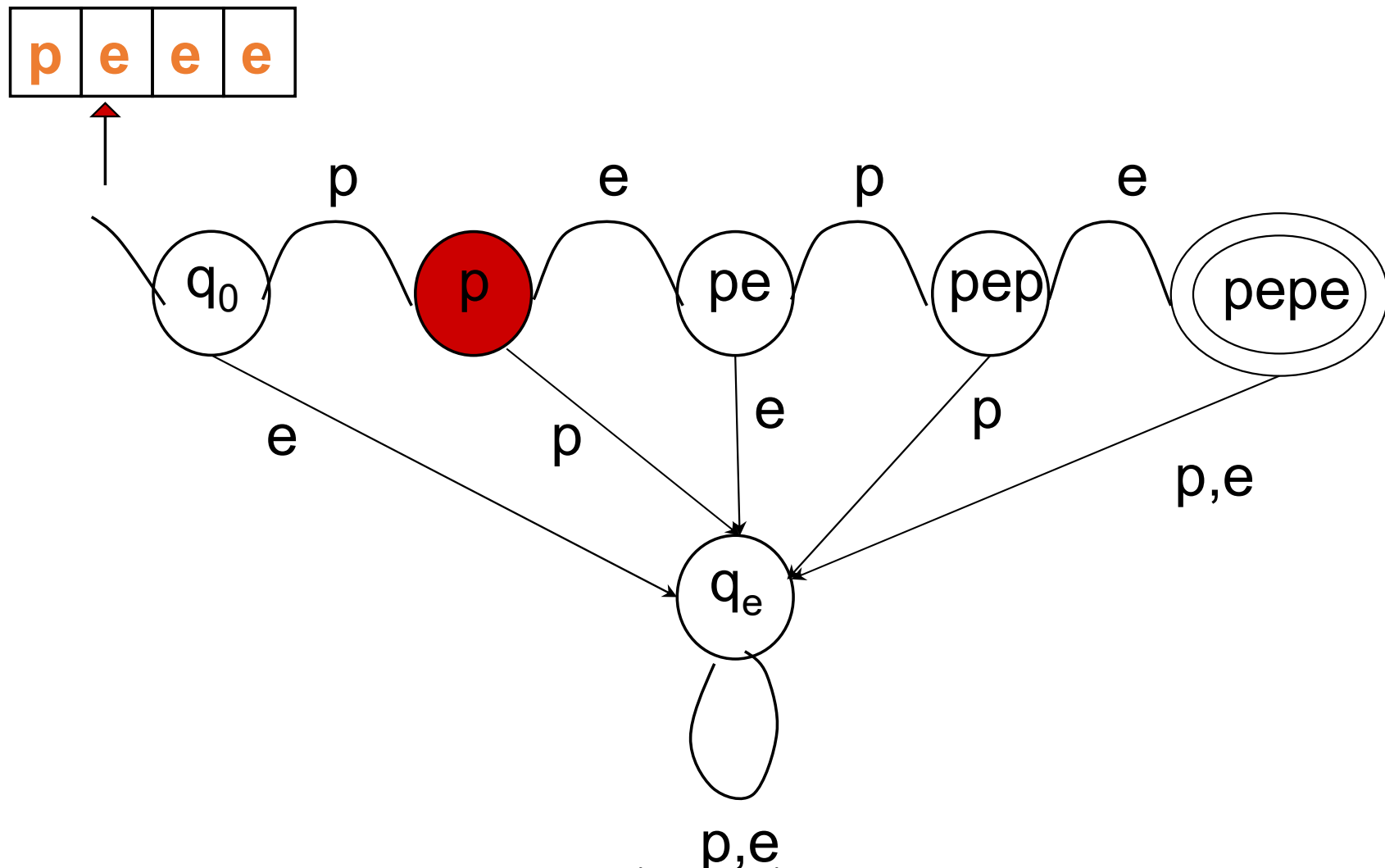
Autómatas finitos

Ejemplo: tratamiento de palabras. Procesamiento de entradas



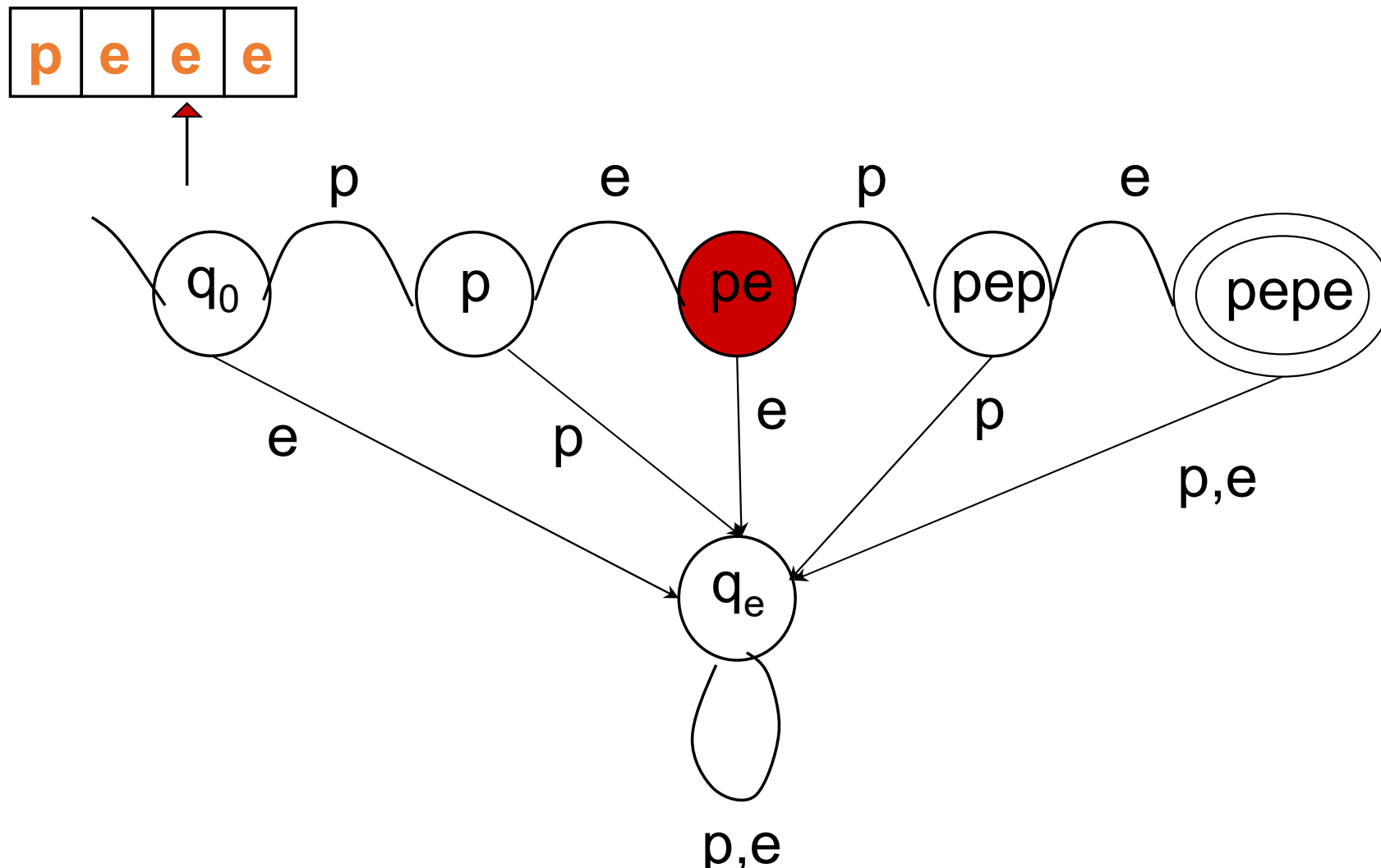
Autómatas finitos

Ejemplo: tratamiento de palabras. Procesamiento de entradas



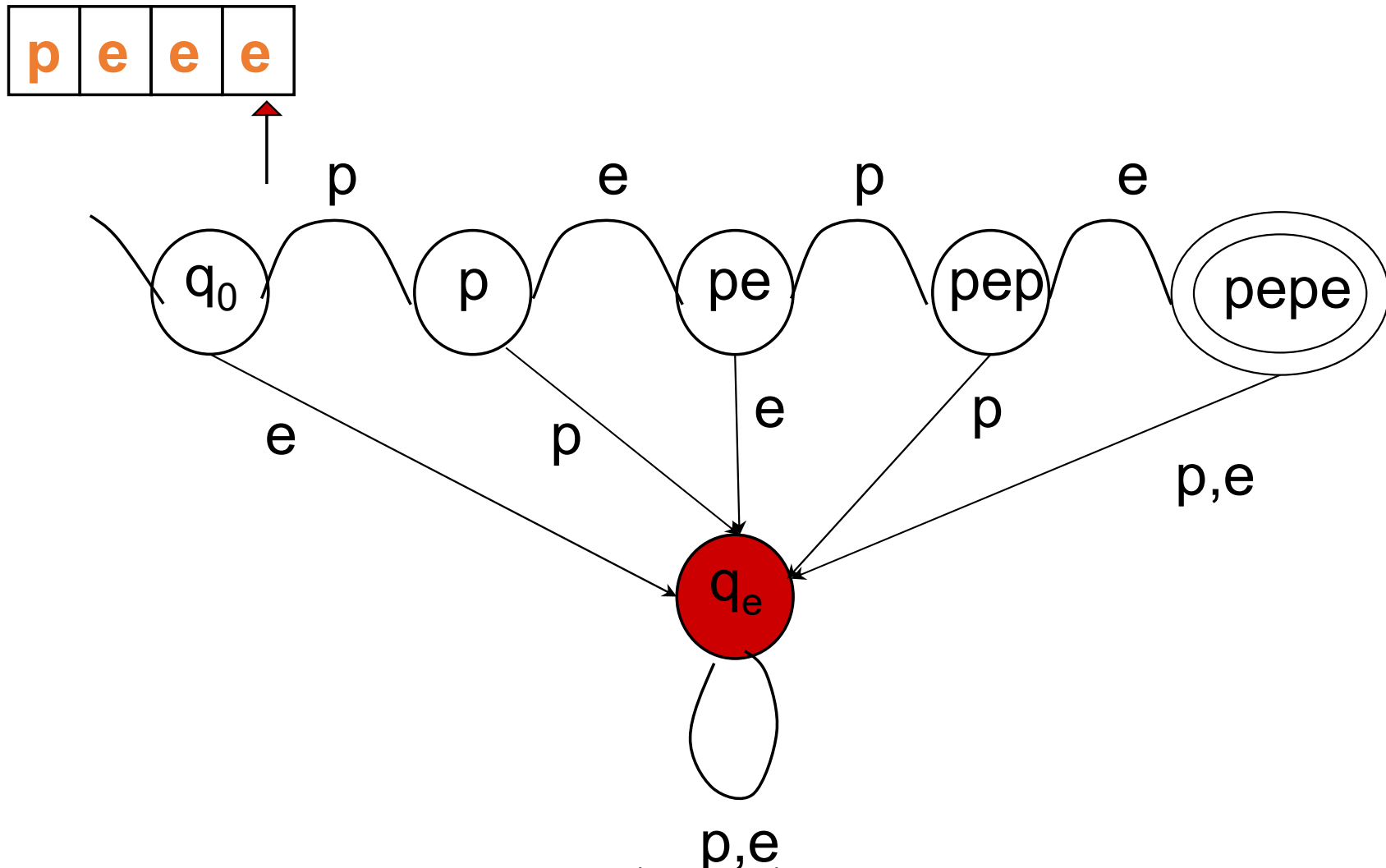
Autómatas finitos

Ejemplo: tratamiento de palabras. Procesamiento de entradas



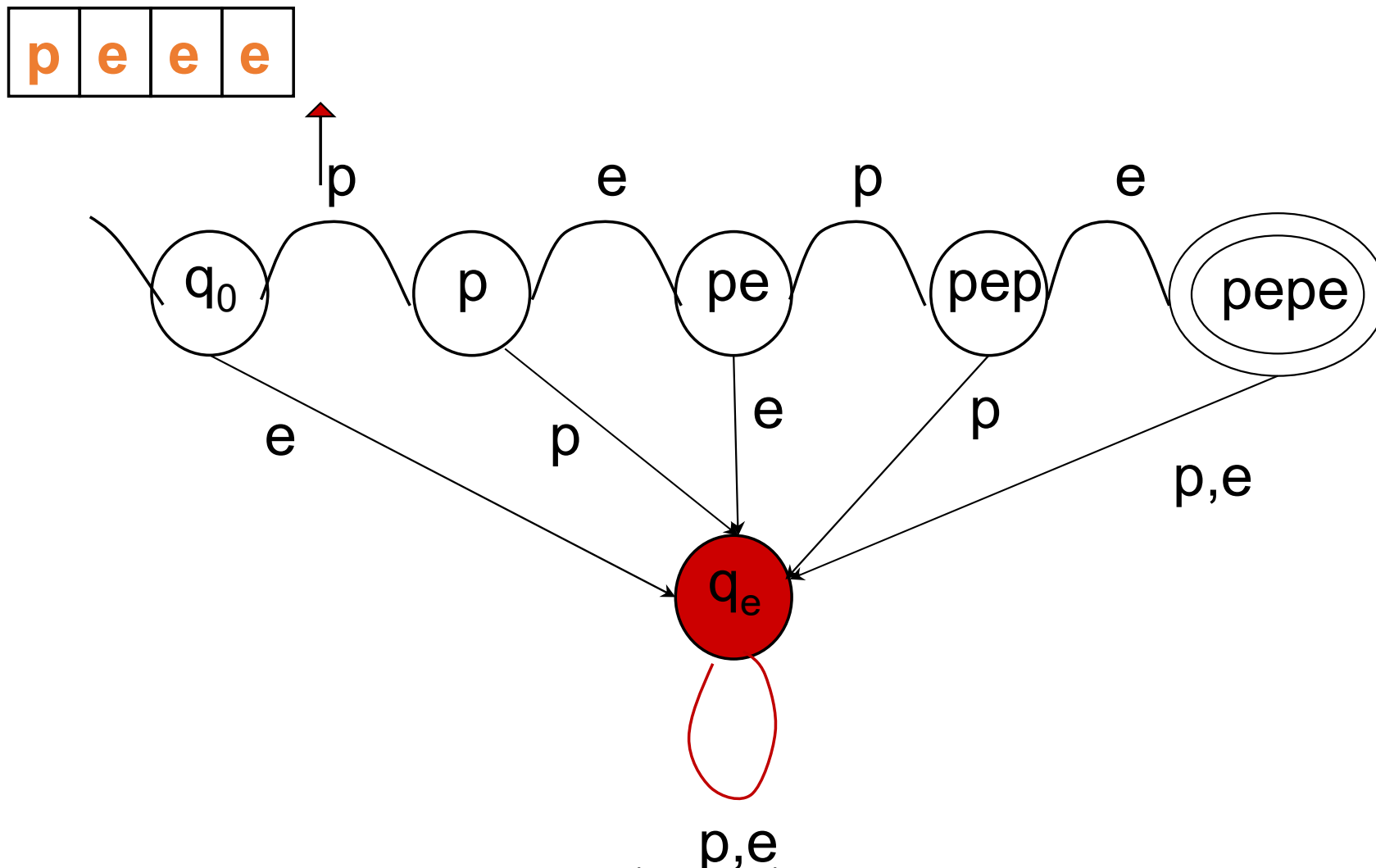
Autómatas finitos

Ejemplo: tratamiento de palabras. Procesamiento de entradas



Autómatas finitos

Ejemplo: tratamiento de palabras. Procesamiento de entradas



Representación de AF's

Más Ejemplos:

$$1) M = (\Sigma, Q, \delta, Q_0, F) = (\{a, b\}, \{q_0, q_1\}, \delta, q_0, \{q_0\})$$

$$\delta : Q \times \Sigma \rightarrow Q$$

$$(q_0, a) \rightarrow q_0$$

$$(q_0, b) \rightarrow q_1$$

$$(q_1, a) \rightarrow q_1$$

$$(q_1, b) \rightarrow q_0$$

L(M)

¿Lenguajes aceptado?



Representación de AF's

Más Ejemplos:

$$1) M = (\Sigma, Q, \delta, Q_0, F) = (\{a, b\}, \{q_0, q_1\}, \delta, q_0, \{q_0\})$$

$$\delta : Q \times \Sigma \rightarrow Q$$

$$(q_0, a) \rightarrow q_0$$

$$(q_0, b) \rightarrow q_1$$

$$(q_1, a) \rightarrow q_1$$

$$(q_1, b) \rightarrow q_0$$

$$L(M) = \{ \omega \in \Sigma^* \mid \omega \text{ tiene un número par de } b\text{'s} \}$$



Representación de AF's

- Para poder definir de una forma más sencilla $L(M)$, podemos cambiar la definición del AF como sigue

$$M = (\Sigma, Q, \delta^*, q_0, F)$$

donde

$$\delta^*: Q \times \Sigma^* \rightarrow Q$$

tal que (recursión)

$$\delta^*(q, \lambda) = q$$

$$\delta^*(q, xw) = \delta^*(\delta(q, x), w)$$

$$\text{con } \delta: Q \times \Sigma \rightarrow Q, x \in \Sigma, w \in \Sigma^*$$

de esta forma la definición de $L(M)$ queda así

$$L(M) = \{w \in \Sigma^* / \delta^*(q_0, w) \in F\}$$



Representación de AF's

- Definición 2.9: Diremos que dos AF M_1 y M_2 son *equivalentes* si $L(M_1) = L(M_2)$
- Observaciones:
 - Se dice que un autómatata finito es más “sencillo” que otro si tiene (con el mismo Σ) menos elementos en Q , es decir, si tiene menos estados.
 - Existen autómatas finitos equivalentes y de distinta “complejidad”
 - Existe un algoritmo capaz de encontrar autómatas finitos mínimos



Estados accesibles

- Teorema 2.2: Sea $M = (\Sigma, Q, \delta^*, q_0, F)$ un AF tal que $|Q| = n$. El estado p es accesible desde el estado $q \Leftrightarrow \exists x \in \Sigma^*, |x| < n$, tal que $\delta^*(q, x) = p$.
- Este teorema se conoce como el **lema del bombeo** (Pumping lemma) y lo utilizaremos más adelante para encontrar lenguajes no regulares
- Nota: este lema utiliza el ***Principio del palomar*** (tema 1)



Estados accesibles

- Demostración:

$\Rightarrow$

Sea p accesible desde q . Entonces, existe $y \in \Sigma^*$ tal que $\delta^*(q, y) = p$.

- Si $|y| < n$, ya está probado.
- Si $|y| > n-1$, entonces el AF pasará por $1 + |y|$ estados desde q hasta p , contando el punto de partida y el final. Como el número total de estados es n , y $|y| > n-1$, se sigue que $1 + |y| > n$, por lo que necesariamente tiene que pasar más de una vez por un mismo estado.



Estados accesibles

Supongamos que el autómata pasa por ese estado repetido en los instantes i y j . Entonces, es posible pasar del estado p al estado q mediante una palabra x , de longitud $|x| < |y|$, formada así:

$$x = P_i \cdot S_j$$

P_i : (prefijo de y hasta la posición i)

S_j : (sufijo de y desde la posición j)

El procedimiento anterior se puede repetir hasta que consigamos pasar desde q hasta p sin que el AF se encuentre dos veces en un mismo estado.

Entonces, $|x| < n$.

$\Leftarrow$ Trivial



Estados accesibles

- Corolario 2.1: Sea $M = (\Sigma, Q, \delta^*, q_0, F)$ un AF tal que $|Q| = n$. Entonces, el lenguaje aceptado por el autómata es no vacío, si y sólo si el autómata finito acepta al menos una palabra $x \in \Sigma^*$ tal que $|x| < n$.

- Demostración:

$\Rightarrow$ Supongamos que el lenguaje aceptado por un autómata finito es no vacío. Entonces, existe al menos una palabra y , tal que $\delta^*(q_0, y) = p \in F$. Luego p es accesible desde q_0 . Y, de acuerdo con el teorema anterior, existirá una palabra $x \in \Sigma^*$, tal que $|x| < n$ y $\delta^*(q_0, x) = p$. Luego x pertenece al lenguaje del autómata finito.

$\Leftarrow$ Trivial



Estados accesibles

- Corolario 2.2: Sea $M = (\Sigma, Q, \delta^*, q_0, F)$ un AF tal que $|Q| = n$. Entonces, el lenguaje aceptado por el autómata es infinito, si y sólo si el autómata finito acepta al menos una palabra $x \in \Sigma^*$ tal que
$$n < |x| < 2n.$$

- Demostración:

$\Rightarrow$ | Ejercicio

$\Leftarrow$ | Trivial



Autómatas Finitos No Deterministas

- Definición: Se llama **autómata finito no determinista** y lo denotaremos por AFND, a la quintupla

$$(Q, \Sigma, q_0, F, \Delta),$$

donde:

- 1) Q es un conjunto finito y no vacío de estados
- 2) Σ es el alfabeto de entrada
- 3) q_0 es un elemento de Q llamado estado inicial
- 4) $F \subseteq Q$ llamado conjunto de estados finales o de aceptación
- 5) Δ es una relación: $(Q \times \Sigma) \times Q$ llamada relación de transición

NOTA: Δ es la principal diferencia entre un AFD y un AFND

- Δ es una relación entre $(Q \times \Sigma)$ y Q



Autómatas Finitos No Deterministas

- Ejemplo: Sea el AFND $(Q, \Sigma, s, F, \Delta)$

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

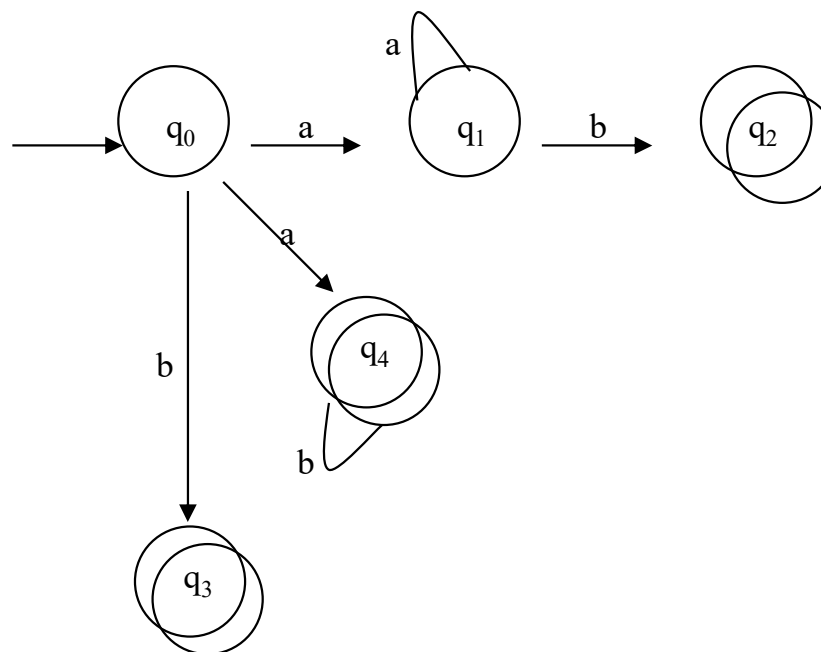
$$F = \{q_2, q_3, q_4\}$$

$$s = q_0$$

$$\Sigma = \{a, b\}$$

Δ :

Δ	a	b
q ₀	{q ₁ , q ₄ }	{q ₃ }
q ₁	∅	{q ₂ }
*q ₂	∅	∅
*q ₃	∅	∅
*q ₄	∅	{q ₄ }



Autómatas Finitos No Deterministas

- Definición 4.2: Si M es un AFND, definimos el lenguaje aceptado por M por medio de:

$$L(M) = \{w \in \Sigma^* / w \text{ es una cadena aceptada por } M\}$$

- Nota: Para que M acepte una palabra basta con que exista un cómputo que lleve al autómata a un estado de aceptación al procesar la palabra completa.
- Teorema 4.1: Sea $M=(Q, \Sigma, s, F, \Delta)$ un AFND. Entonces existe un AFD M' equivalente.
 - Demostración: Algoritmo para la transición de un AFND a un AFD



Autómatas Finitos No Deterministas

ALGORITMO PARA LA TRANSICIÓN DE UN AFND A UN AFD CONEXO

Sean $A = \{\{q_0\}\}$, $R = \wp(Q) - A$, $A' = A$

Dónde q_0 es el estado inicial del AFND y $\wp(Q)$ es el conjunto partes de Q .

repetir

$\forall q \in A'$ repetir

$\forall x \in \Sigma$ repetir

$\Delta(x, q) = \delta(x, q) = p$

 Si $\Delta(x, q) = p \wedge p \in R$ entonces

$A = A \cup \{p\}$

$R = R \setminus \{p\}$

 fin si

 fin repetir

fin repetir

fin repetir



Autómatas Finitos No Deterministas

- Equivalencia de AFND y AFD
 - La definición de equivalencia para AFD se puede extender para todos los autómatas finitos (AFD y AFND):
 - Dos autómatas M y M' son equivalentes si
$$L(M) = L(M')$$



Autómatas Finitos No Deterministas

Ejemplo1: Considérese el AFND

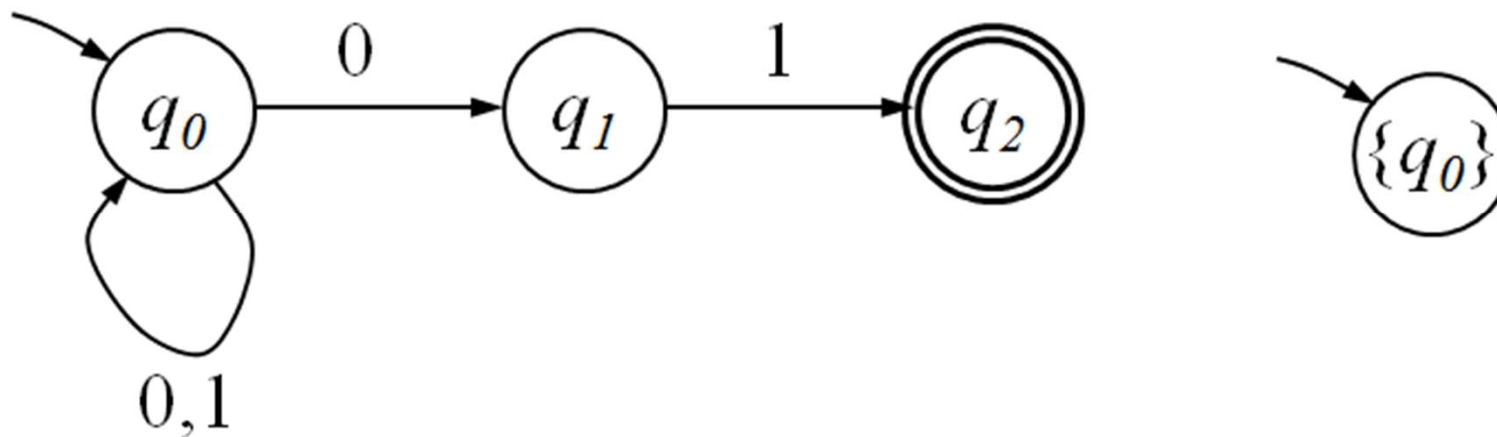
$$A=(Q=\{q_0, q_1, q_2\}, \Sigma=\{0,1\}, \delta, q_0, F=\{q_2\})$$

donde la tabla de δ es el siguiente:

	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	Φ	$\{^*q_2\}$
*q_2	Φ	Φ

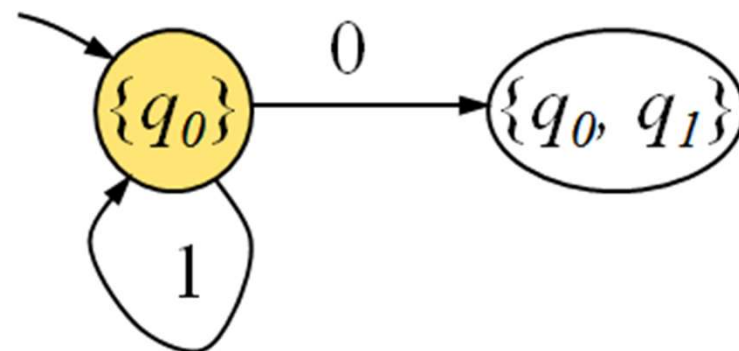
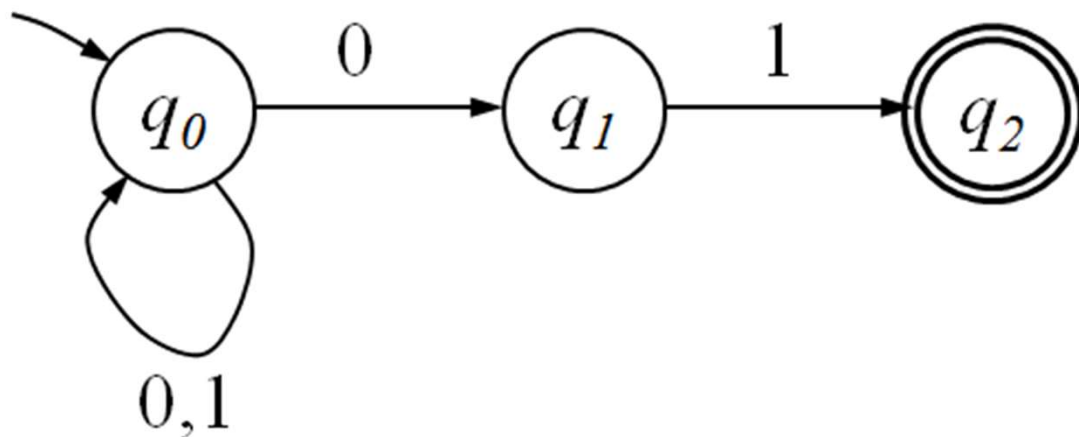
Autómatas Finitos No Deterministas

Ejemplo 1: Construcción del AFD Equivalente



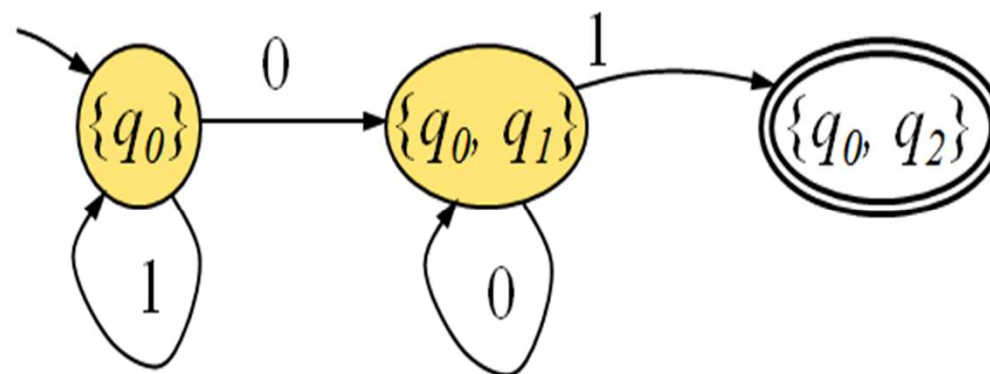
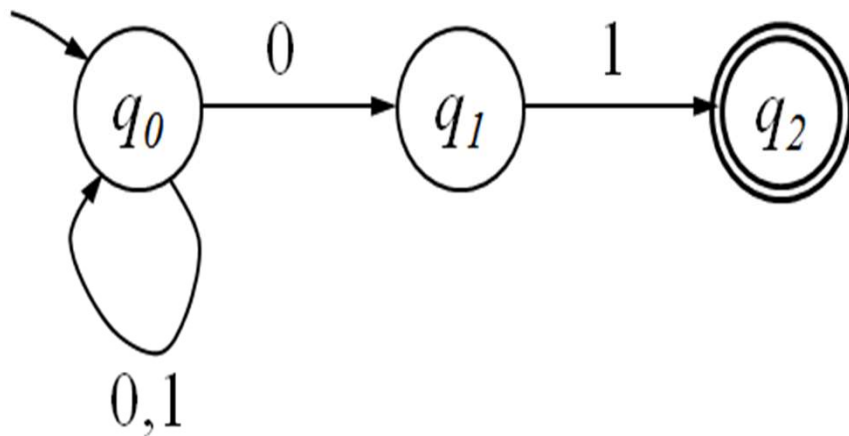
Autómatas Finitos No Deterministas

Ejemplo 1: Construcción del AFD Equivalente



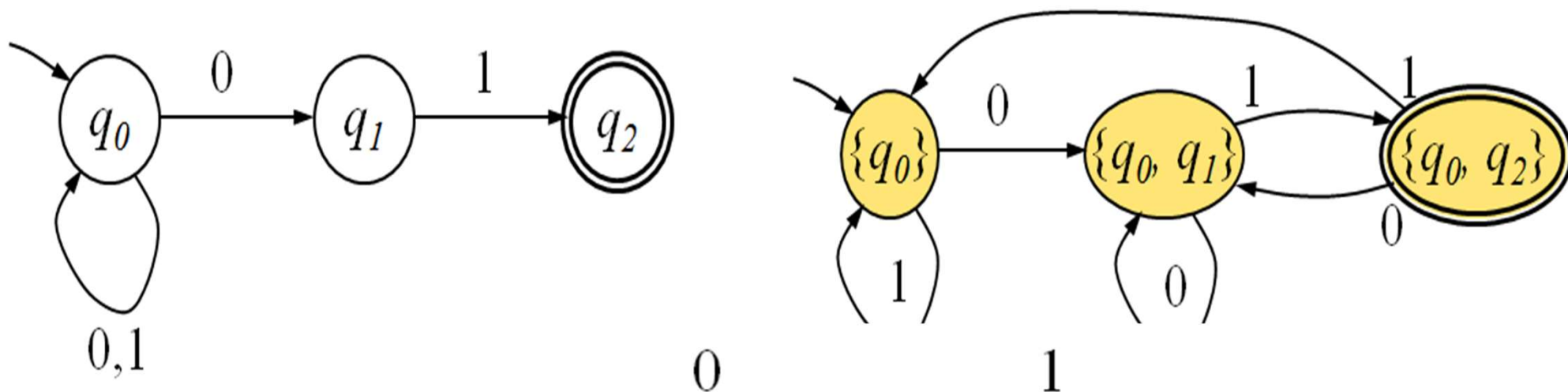
Autómatas Finitos No Deterministas

Ejemplo 1: Construcción del AFD Equivalente



Autómatas Finitos No Deterministas

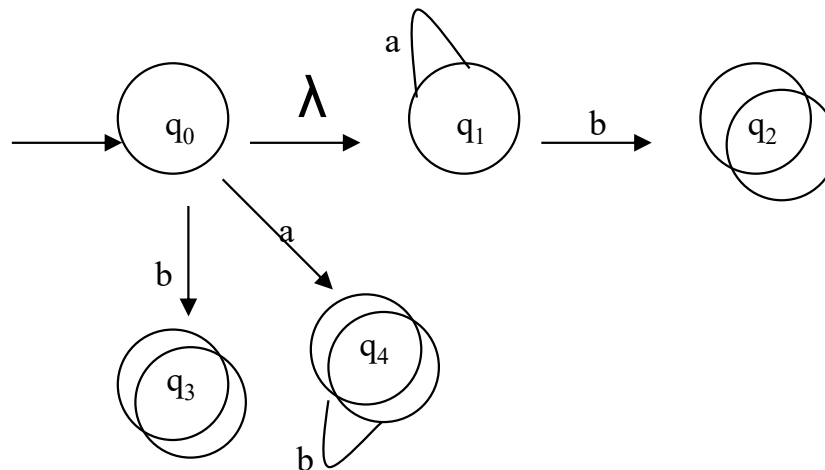
Ejemplo 1: Construcción del AFD Equivalente



	0	1
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$*\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$

Autómatas Finitos No Deterministas

- λ -transiciones
 - Transiciones de un estado a otro que no dependen de ninguna entrada (no consumen ningún símbolo de entrada)



- A un AFND con λ -transiciones lo denominaremos:
 λ -AFND



Autómatas Finitos No Deterministas

- Definición 4.3: Se llama **autómata finito no determinista con λ -transiciones** y lo denotaremos por λ -AFND, a la quintupla

$(Q, \Sigma, q_0, F, \Delta)$, donde:

1) Q, Σ, q_0, F iguales que AFND

2) relación de transición:

$$\Delta \subseteq (Q \times (\Sigma \cup \{\lambda\})) \times Q$$

- Definición 4.4: Si M es un λ -AFND, definimos el lenguaje aceptado por M por medio de:

$$L(M) = \{x \in \Sigma^* / \exists p \in F, p \in \Delta^*(q_0, x) \}$$



Autómatas Finitos No Deterministas

- Definición 4.5: Se llama **λ -cierre** de un estado $q \in Q$ y lo denotaremos por **$\lambda\text{-c}(q)$** al conjunto de estados a los que se transitan usando únicamente λ -transiciones:

$$\lambda\text{-c}(q) = \{p \in Q / p \in \Delta^*(q, \lambda)\}$$

- Definición 4.6: Se llama **distancia** de un estado $q \in Q$ respecto a un símbolo $x \in \Sigma$ y lo denotaremos por **$d(q, x)$** al conjunto de estados a los que se transitan sin usar ninguna λ -transición:

$$d(q, x) = \{p \in Q / p \in \Delta(q, x) \wedge p \notin \Delta^*(q, \lambda)\}$$



Autómatas Finitos No Deterministas

ALGORITMO PARA LA TRANSICIÓN DE UN λ -AFND A UN AFD CONEXO

Sean
 $A = \{\{q_0\}\}$ $R = \wp(Q) - A$ $A' = A$
 ,

Dónde q_0 es el estado inicial del AFND y $\wp(Q)$ es el conjunto partes de Q .
repetir

$\forall q \in A'$ repetir

$\forall x \in \Sigma$ repetir

$\delta(x, q) = \lambda - c(d(\lambda - c(q), x)) = p$

 Si $\Delta(x, q) = p \wedge p \in R$ entonces

$A = A \cup \{p\}$

$R = R \setminus \{p\}$

 fin si

 fin repetir

fin repetir

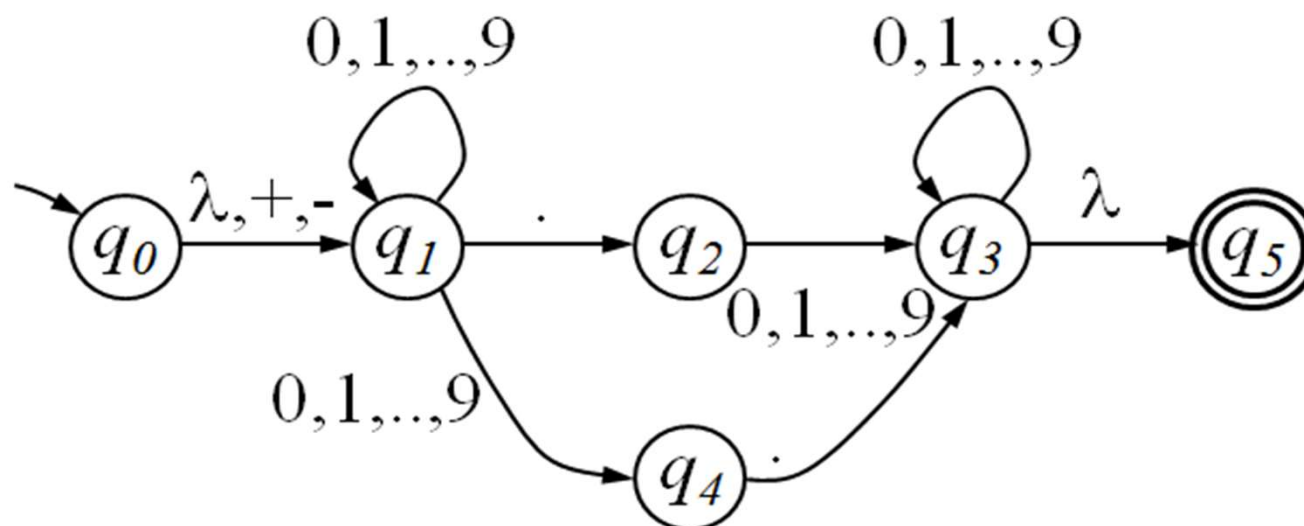
fin repetir



Autómatas Finitos No Deterministas

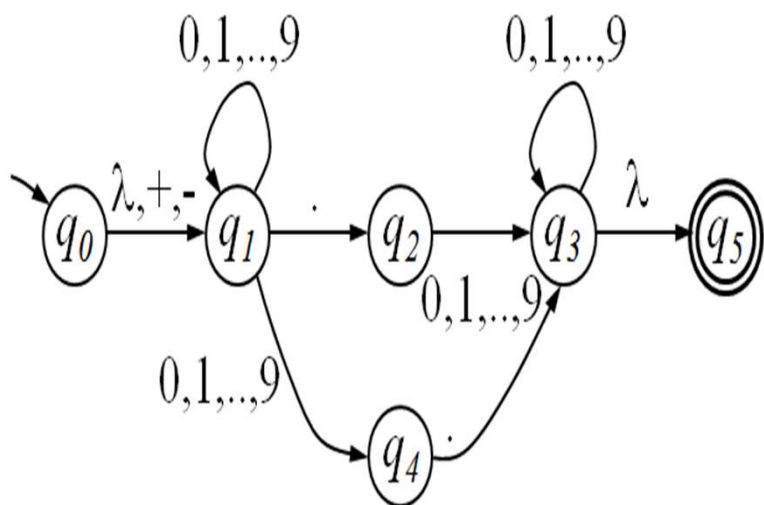
Ejemplo: $A=(Q=\{q_0,q_1,q_2,q_3,q_4,q_5\},\Sigma=\{0,1,\dots,9,+,-,.\},\delta,q_0,F=\{q_5\})$

Donde δ queda representado en el diagrama:



Autómatas Finitos No Deterministas

Ejemplo: cálculo de las lambda-cerraduras



$$\lambda - c(q_0) = \{q_0, q_1\}$$

$$\lambda - c(q_1) = \{q_1\}$$

$$\lambda - c(q_2) = \{q_2\}$$

$$\lambda - c(q_3) = \{q_3, q_5\}$$

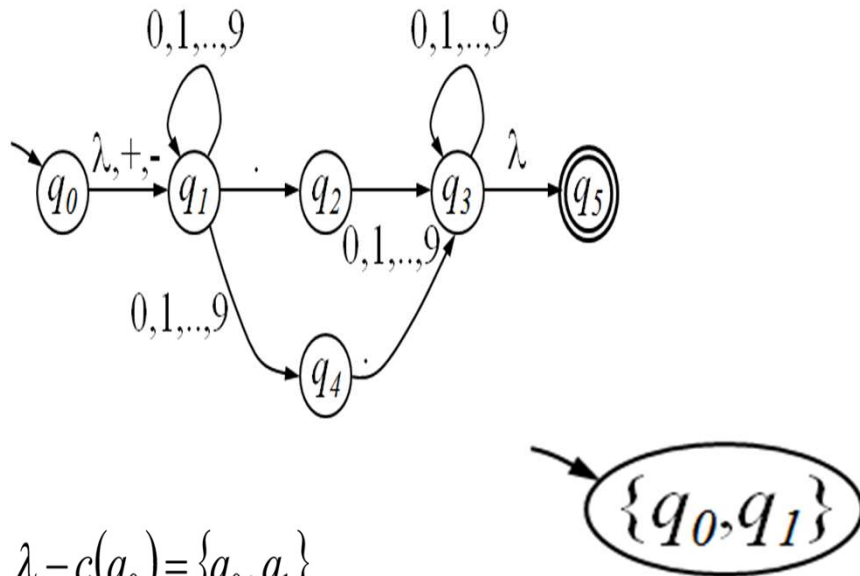
$$\lambda - c(q_4) = \{q_4\}$$

$$\lambda - c(q_5) = \{q_5\}$$



Autómatas Finitos No Deterministas

Ejemplo: cálculo del AFD equivalente



$$\lambda - c(q_0) = \{q_0, q_1\}$$

$$\lambda - c(q_1) = \{q_1\}$$

$$\lambda - c(q_2) = \{q_2\}$$

$$\lambda - c(q_3) = \{q_3, q_5\}$$

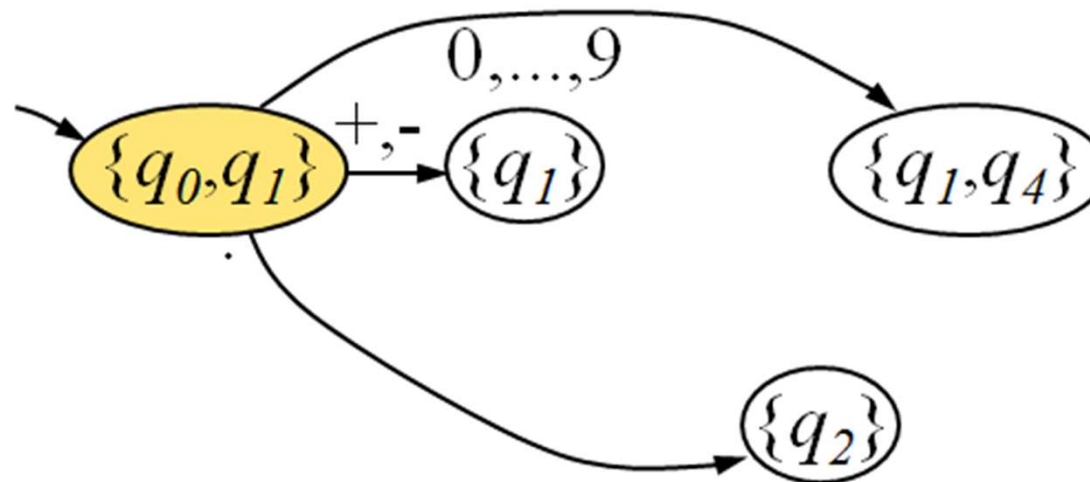
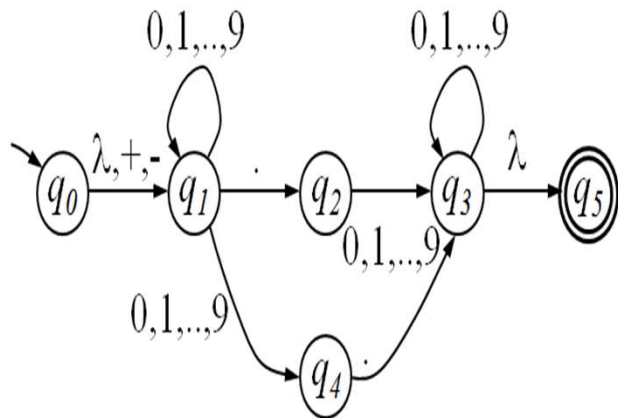
$$\lambda - c(q_4) = \{q_4\}$$

$$\lambda - c(q_5) = \{q_5\}$$



Autómatas Finitos No Deterministas

Ejemplo: cálculo del AFD equivalente



$$\lambda - c(q_0) = \{q_0, q_1\}$$

$$\lambda - c(q_1) = \{q_1\}$$

$$\lambda - c(q_2) = \{q_2\}$$

$$\lambda - c(q_3) = \{q_3, q_5\}$$

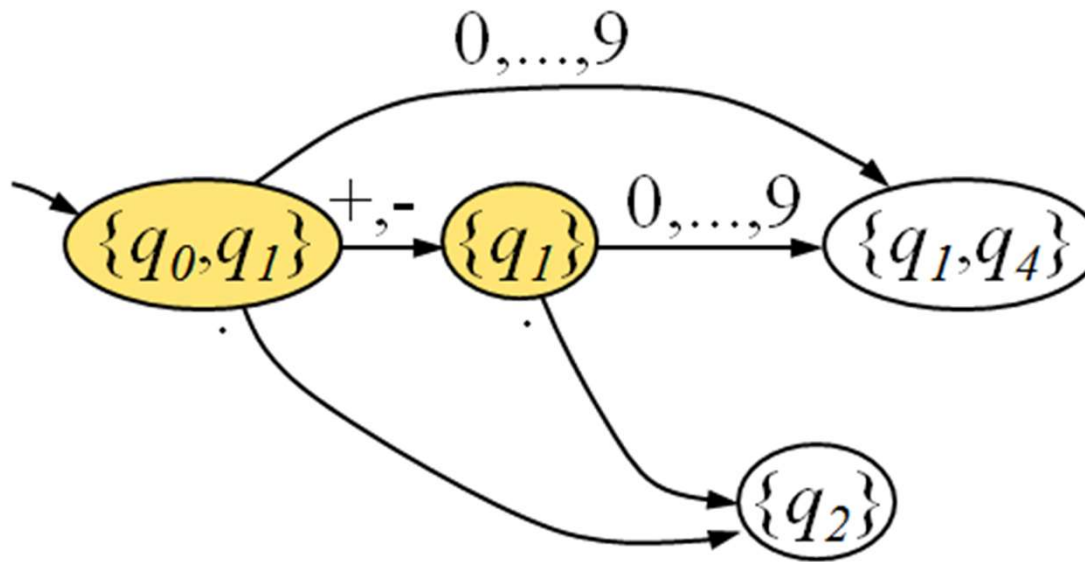
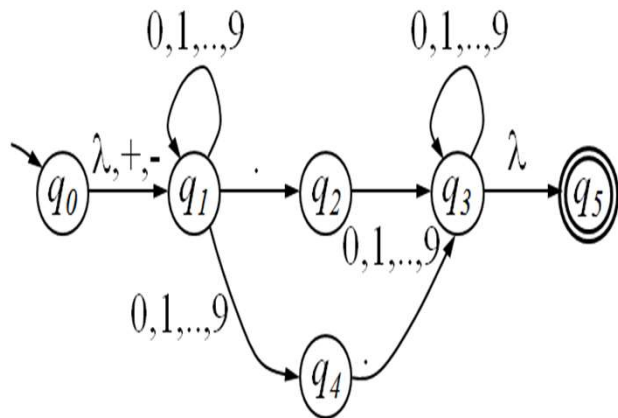
$$\lambda - c(q_4) = \{q_4\}$$

$$\lambda - c(q_5) = \{q_5\}$$



Autómatas Finitos No Deterministas

Ejemplo: cálculo del AFD equivalente



$$\lambda - c(q_0) = \{q_0, q_1\}$$

$$\lambda - c(q_1) = \{q_1\}$$

$$\lambda - c(q_2) = \{q_2\}$$

$$\lambda - c(q_3) = \{q_3, q_5\}$$

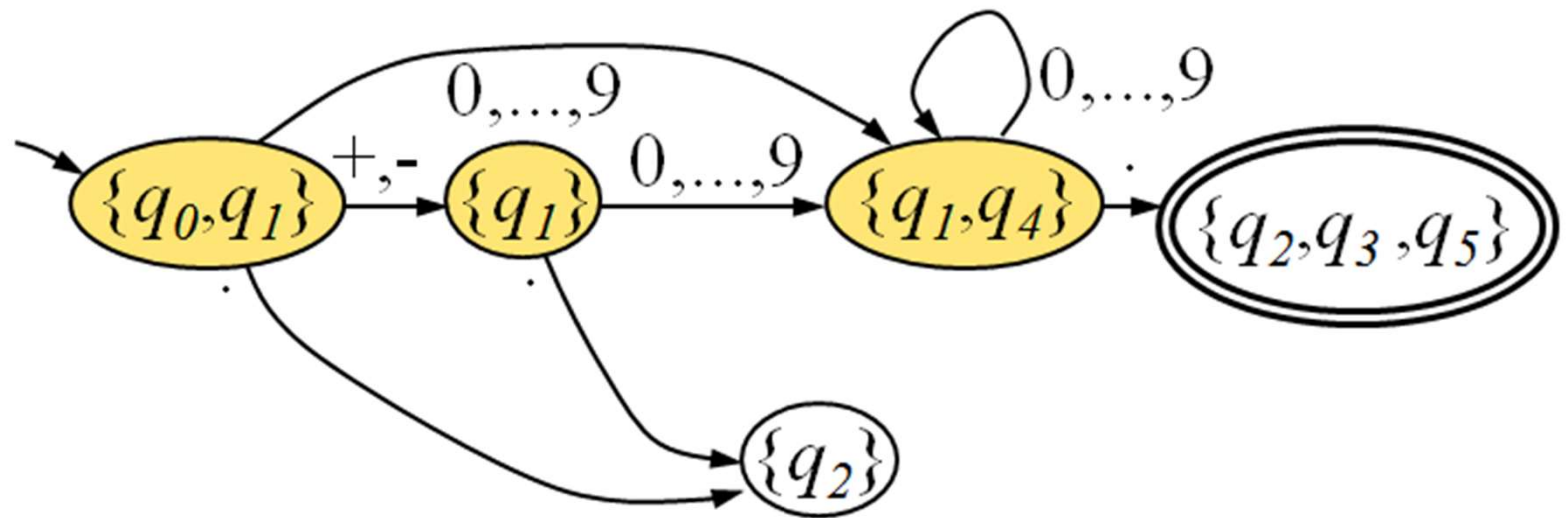
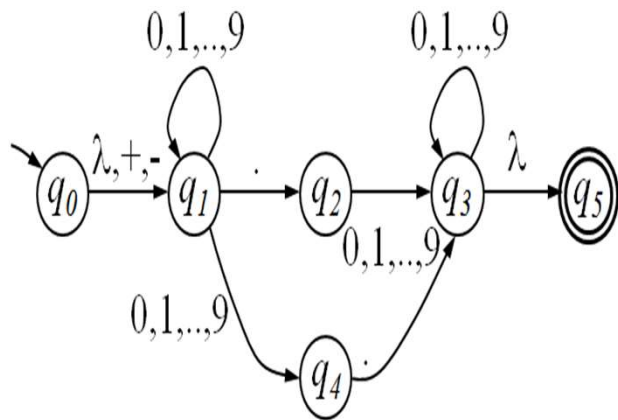
$$\lambda - c(q_4) = \{q_4\}$$

$$\lambda - c(q_5) = \{q_5\}$$



Autómatas Finitos No Deterministas

Ejemplo: cálculo del AFD equivalente



$$\lambda - c(q_0) = \{q_0, q_1\}$$

$$\lambda - c(q_1) = \{q_1\}$$

$$\lambda - c(q_2) = \{q_2\}$$

$$\lambda - c(q_3) = \{q_3, q_5\}$$

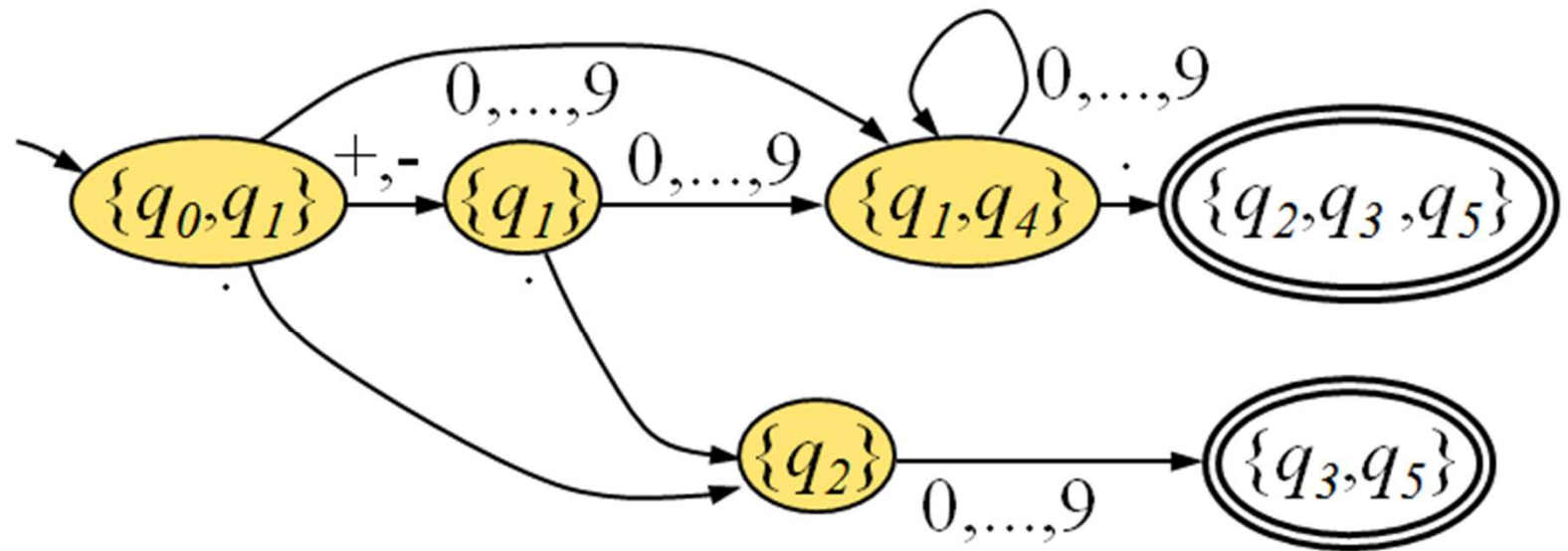
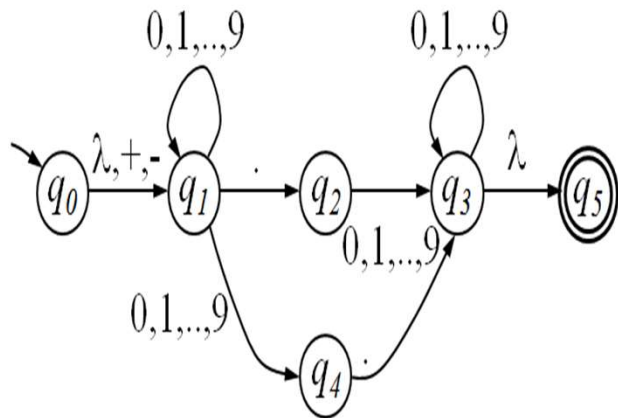
$$\lambda - c(q_4) = \{q_4\}$$

$$\lambda - c(q_5) = \{q_5\}$$



Autómatas Finitos No Deterministas

Ejemplo: cálculo del AFD equivalente



$$\lambda - c(q_0) = \{q_0, q_1\}$$

$$\lambda - c(q_1) = \{q_1\}$$

$$\lambda - c(q_2) = \{q_2\}$$

$$\lambda - c(q_3) = \{q_3, q_5\}$$

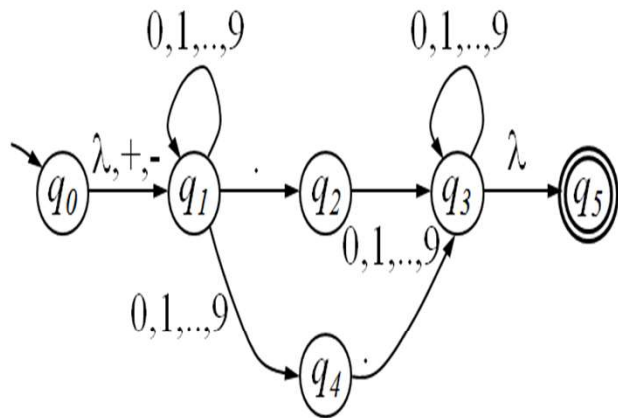
$$\lambda - c(q_4) = \{q_4\}$$

$$\lambda - c(q_5) = \{q_5\}$$



Autómatas Finitos No Deterministas

Ejemplo: cálculo del AFD equivalente (se omite el estado sumidero)



$$\lambda - c(q_0) = \{q_0, q_1\}$$

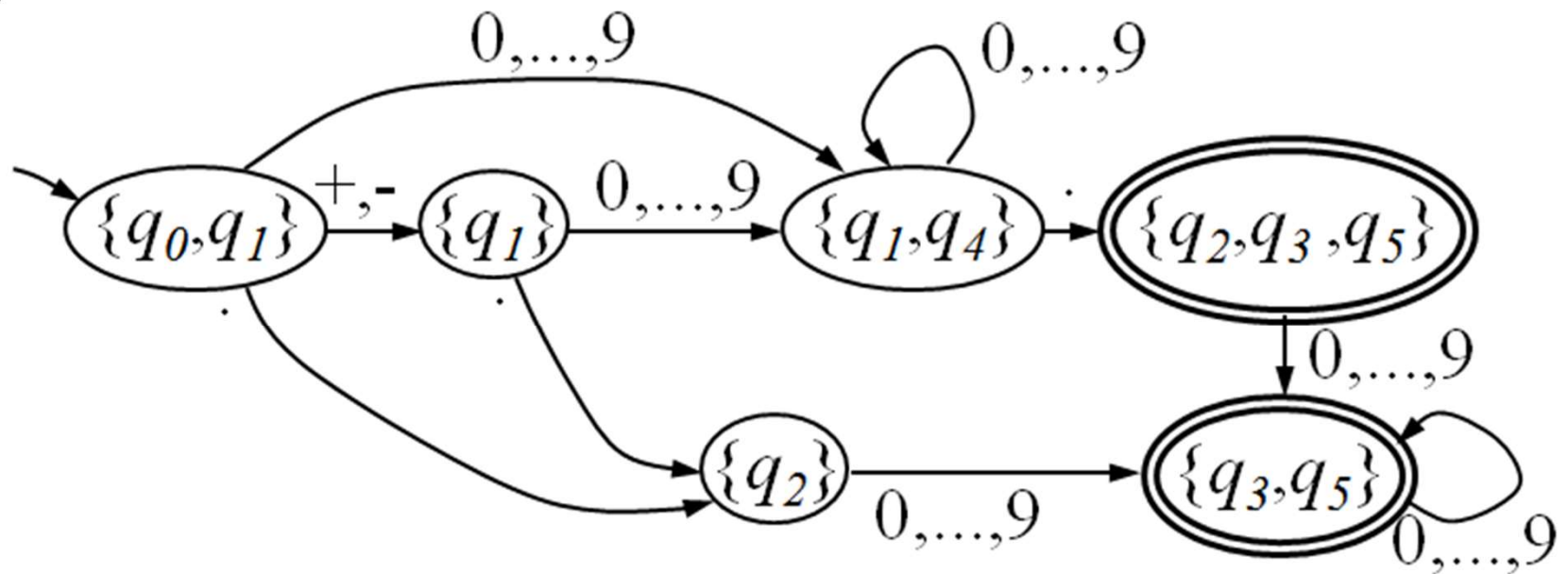
$$\lambda - c(q_1) = \{q_1\}$$

$$\lambda - c(q_2) = \{q_2\}$$

$$\lambda - c(q_3) = \{q_3, q_5\}$$

$$\lambda - c(q_4) = \{q_4\}$$

$$\lambda - c(q_5) = \{q_5\}$$



Autómatas Finitos No Deterministas

- Definición 4.7: Se llama **autómata finito no determinista genérico** y lo denotaremos por G-AFND, a la quintupla

$(Q, \Sigma, q_0, F, \Delta)$, donde:

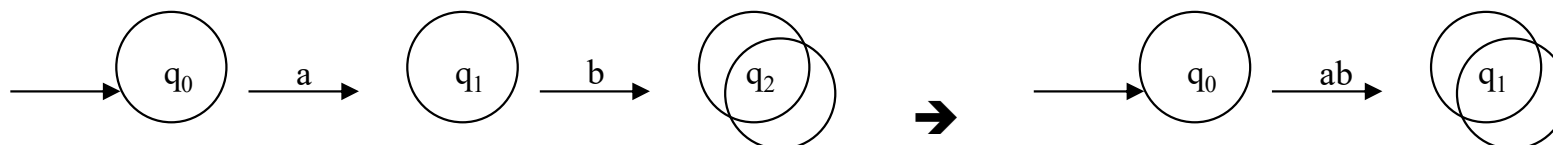
1) Q, Σ, q_0, F iguales que AFND

2) relación de transición:

$$\Delta \subseteq (Q \times \Sigma^*) \times Q$$

- Ejemplo:

$$\Delta(q_0, \text{"ab"}) = q_1$$



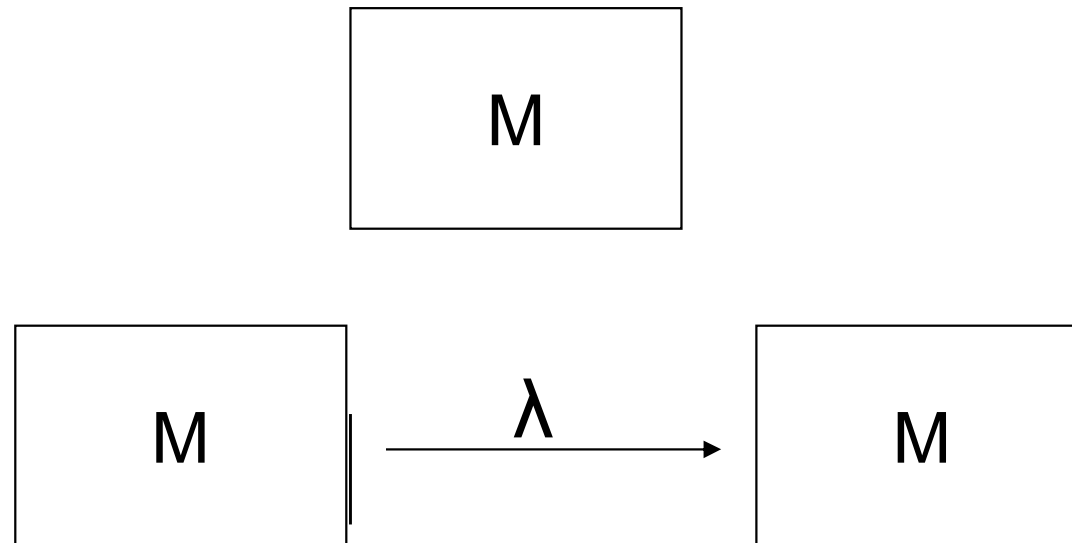
Autómatas Finitos No Deterministas

- Equivalencia de G-AFND y λ -AFND
 - La definición de equivalencia para AFD y AFND se puede extender para los autómatas finitos G-AFND y λ -AFND
 - Dos autómatas M y M' son equivalentes si $L(M) = L(M')$
- Equivalencia entre G-AFND y λ -AFND
 - Todo autómata G-AFND se puede representar mediante un λ -AFND y viceversa
 - Demostración: Ejercicio



Máquinas secuenciales

- Teorema 4.2 : Sea M un AF. Entonces existe un AF M' equivalente a M con un único estado de aceptación.
- Notación: Se puede representar un AF como una CAJA con un círculo inicial y un cuadrado al final:



Máquinas secuenciales

- Teorema 4.2: Teorema de Síntesis de Kleene: Todo lenguaje caracterizado por una ER es reconocido por un AFND

ER $\Rightarrow$ AFND

- Demostración: Nos basamos en la definición de ER

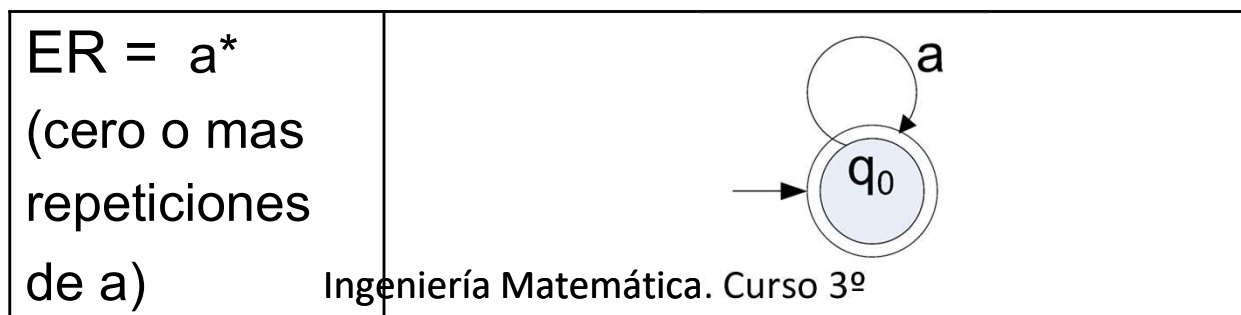
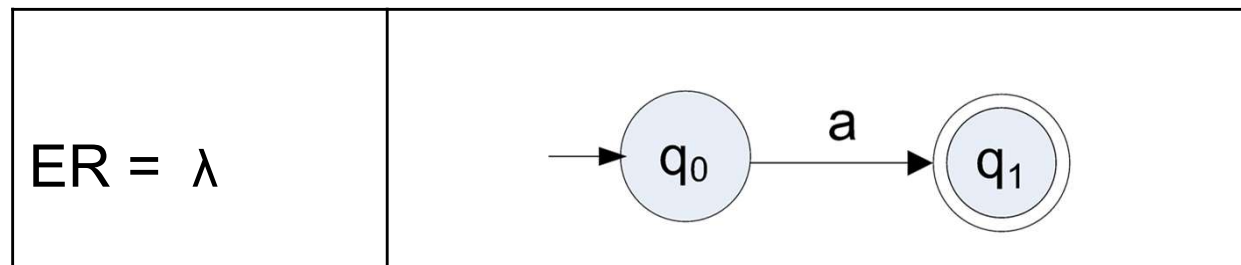
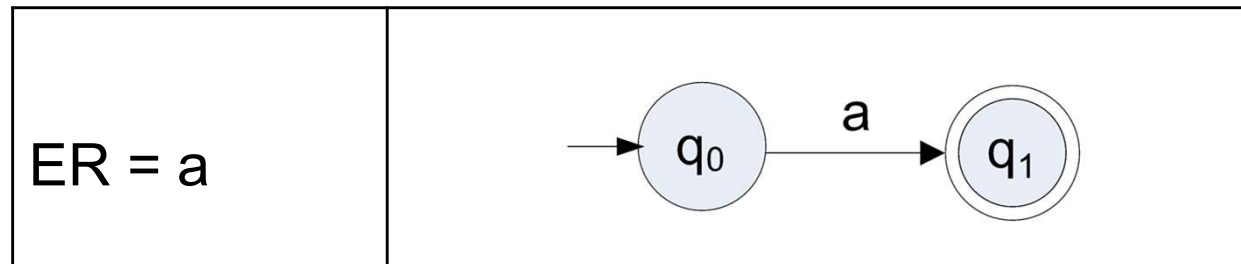
Son expresiones regulares:

- a) $\emptyset$
- b) λ
- c) a
- d) $\alpha + \beta$
- e) $\alpha \cdot \beta$
- f) α^*



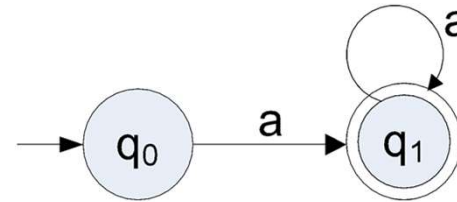
Conceptos de AFND

- **Equivalencias entre Expresiones Regulares básicas y autómatas finitos:** Vamos a ver las expresiones regulares básicas: repetición en sus distintas variantes, alternativa y concatenación y su equivalente AF.

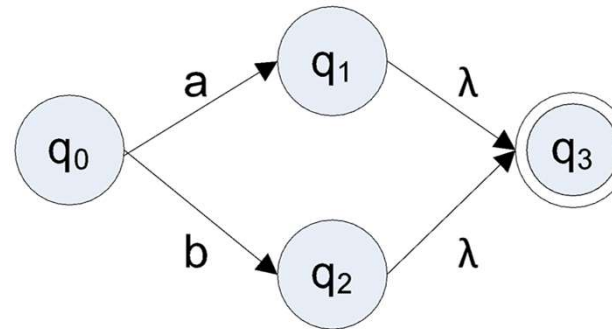


Conceptos de AFND

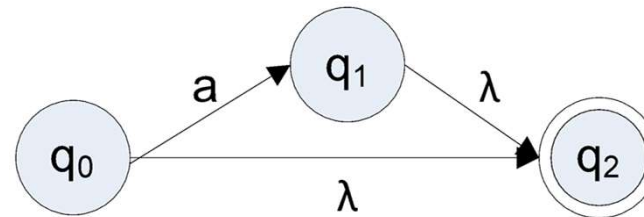
$ER = a^+ = aa^*$
(una o mas repeticiones de a)



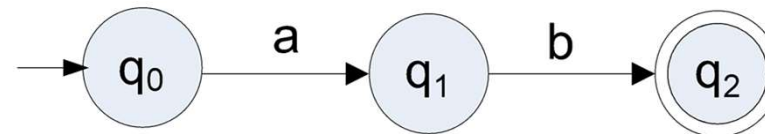
$ER = a|b$
(Alternativa)



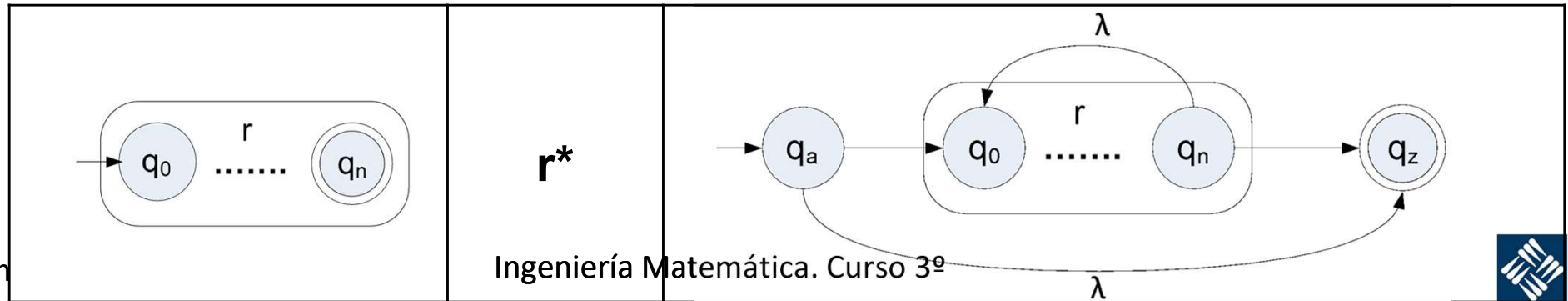
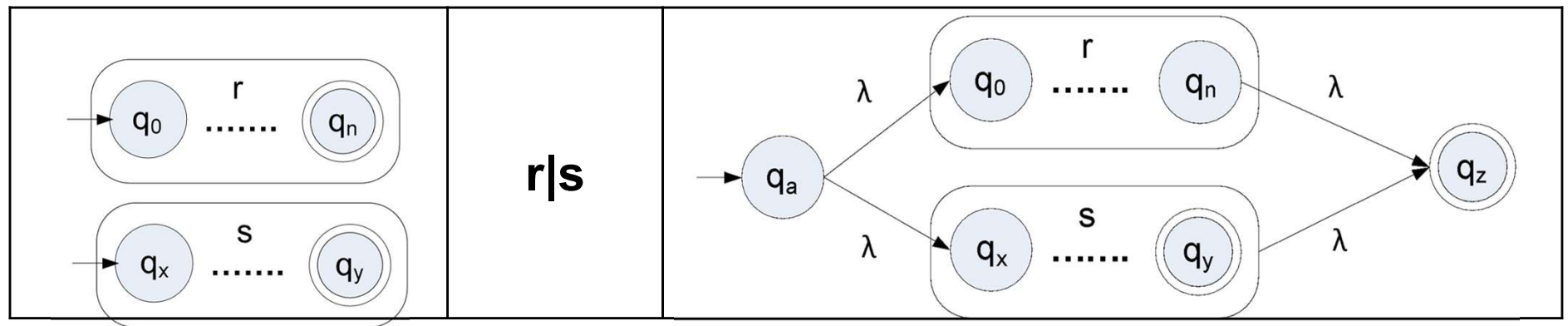
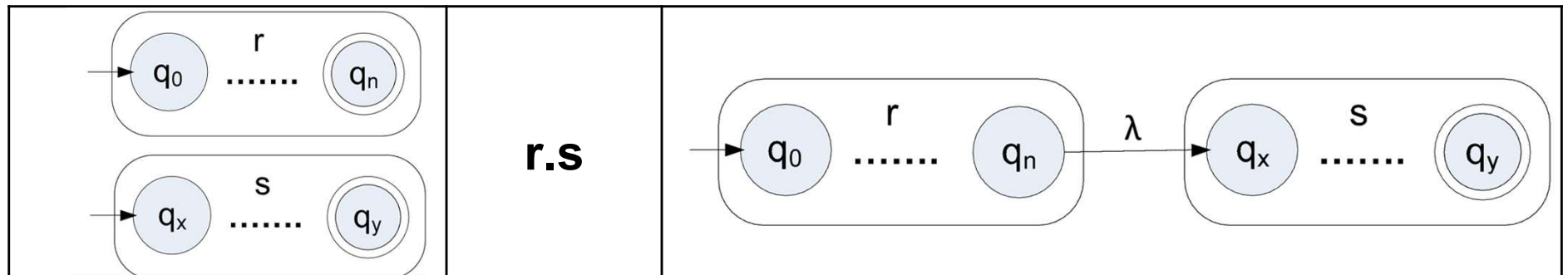
$ER = a? = a|\lambda$
(cero o una instancia de a)



$ER = ab$
(Concatenación)



Diagramas de Thomson

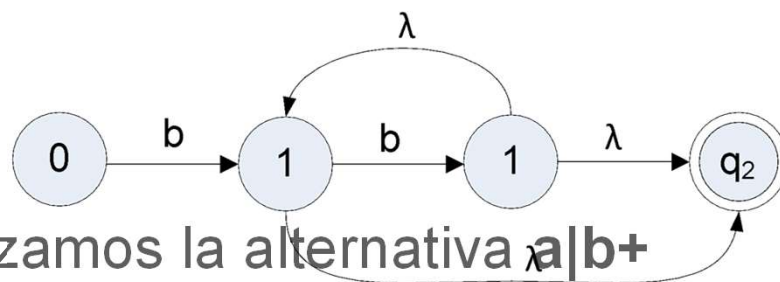


Conversión de una ER en un AFN

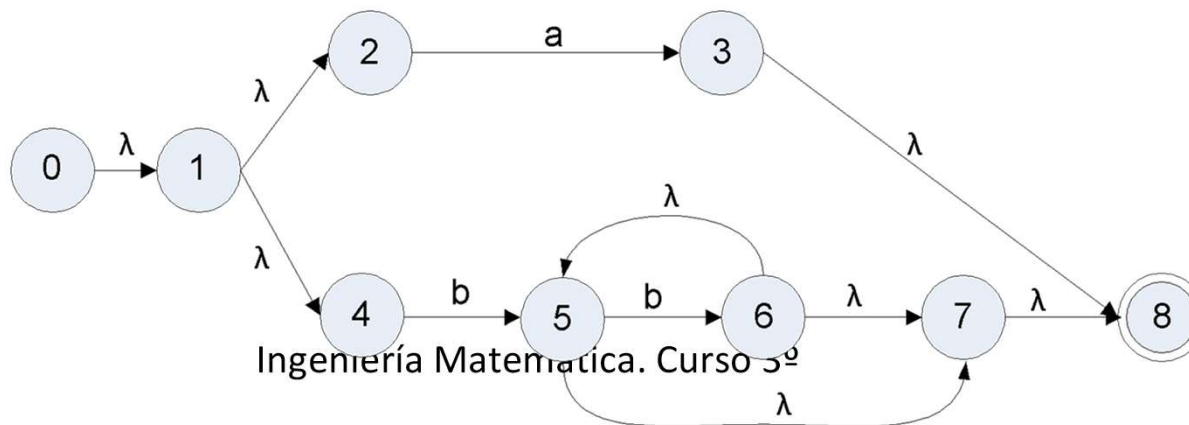
- Ejemplo: Construir el AFN para la cadena $(a|b^+)b^?$
- **Paso 1:** Hacemos la construcción de Thomson para a .



- **Paso 2:** Obtenemos la construcción de Thomson de b^+ (recordemos que es igual que bb^*).

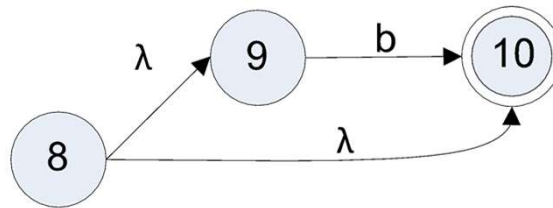


- **Paso 3:** Realizamos la alternativa $a|b^+$

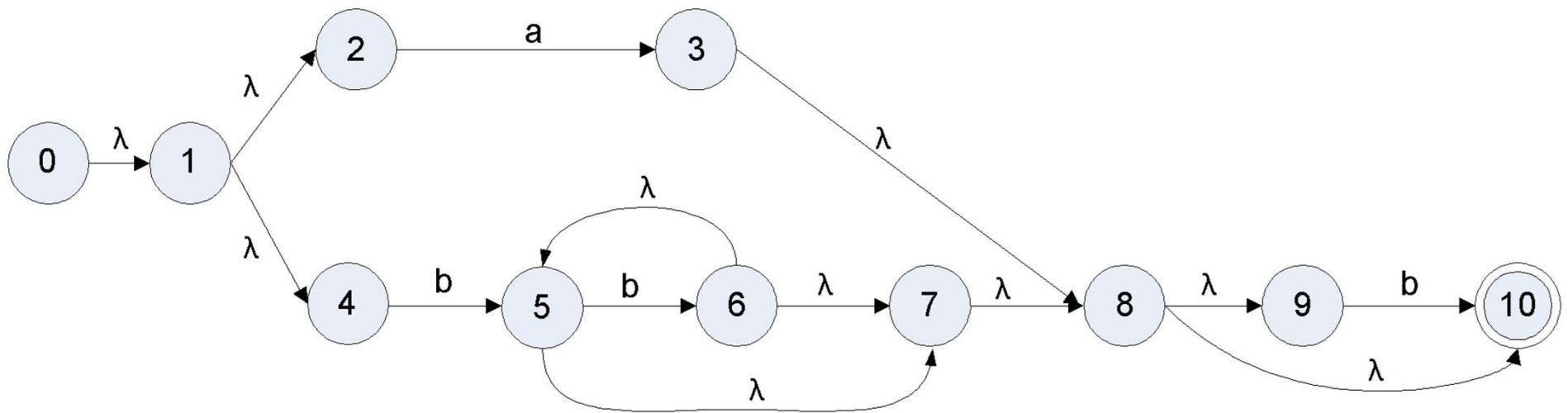


Conversión de una ER en un AFN

- **Paso 4:** Realizamos $b^?$, que indica cero o una instancia de b



- **Paso 5:** Lo unimos todo para obtener $(a|b^+)b^?$



Autómatas Finitos Deterministas

Conversión de un AFN en un AFD

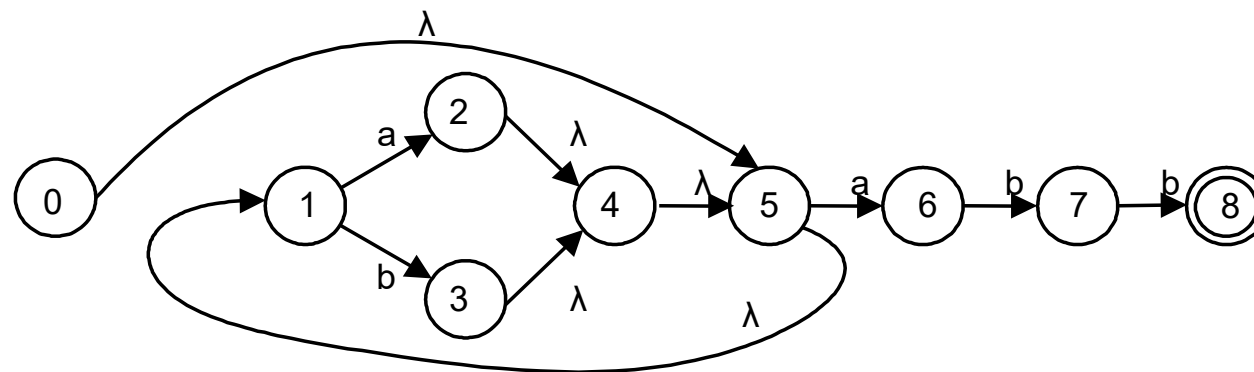
- Implementar un AFD es más sencillo que un AFN.
 - En cada estado del autómata, existe una única transición posible para un símbolo determinado.
 - No existen λ -transiciones.
 - $\Rightarrow$ No hay que probar varias posibilidades en cada estado.



Autómatas Finitos Deterministas

Conversión de un AFN en un AFD

- Ejemplo AFN para $(a|b)^* abb$.
 - En lugar de adivinar qué λ -transición debemos tomar, diremos que el AFN puede tomar cualquiera, y formamos un estado conjunto: $\{0, 5, 1\}$ (cierre- $\lambda(\{1\})$).
 - Ahora tomamos la transición para el carácter a. Desde el estado 1, podemos alcanzar el 2; y desde el estado 5, podemos alcanzar el 6. Así, tenemos el estado $\{2, 6\}$.
 - Si computamos el cierre- $\lambda(\{2,6\})$, obtenemos $\{1, 2, 4, 5, 6\}$.



Autómatas Finitos Deterministas

Conversión de un AFN en un AFD

- Definición formal de cierre- λ .
 - Sea transición(s, c) el conjunto de todos los estados del AFN alcanzables desde s mediante transiciones directas etiquetadas con c .
 - Para un conjunto de estados S , cierre- $\lambda(S)$ es el conjunto de estados que pueden ser alcanzados desde S sin consumir ningún símbolo de entrada.



Autómatas Finitos Deterministas

Conversión de un AFN en un AFD

- Cálculo de cierre- $\lambda(S)$.
 - meter todos los elementos de S en una pila
 - inicializar cierre- $\lambda(S)$ a S
 - mientras** la pila no esté vacía **hacer**
 - sacar s, el elemento tope de la pila
 - para** cada estado t alcanzable desde s mediante λ **hacer**
 - si** t no está en cierre- $\lambda(S)$ **entonces**
 - añadir t a cierre- $\lambda(S)$
 - meter t en la pila
 - fin**
- fin**



Autómatas Finitos Deterministas

Conversión de un AFN en un AFD

- Algoritmo para convertir un AFN en AFD.
 - Entrada: AFN N .
 - Salida: AFD D .
 - Método: Se construye una tabla de transiciones $\text{tran}D$ para D , de manera que $\text{tran}D$ simula “en paralelo” todos los posibles movimientos que se pueden dar en N ante una determinada cadena de entrada.
 - Además del cierre- λ , se utiliza la operación $\text{mueve}(T, a)$, que dará como resultado un conjunto de estados del AFN hacia los cuales existe una transición con el símbolo a desde algún estado $s \in T$.
 - Tomamos 0 como el estado inicial del AFN.



Autómatas Finitos Deterministas

Conversión de un AFN en un AFD

– Pseudocódigo:

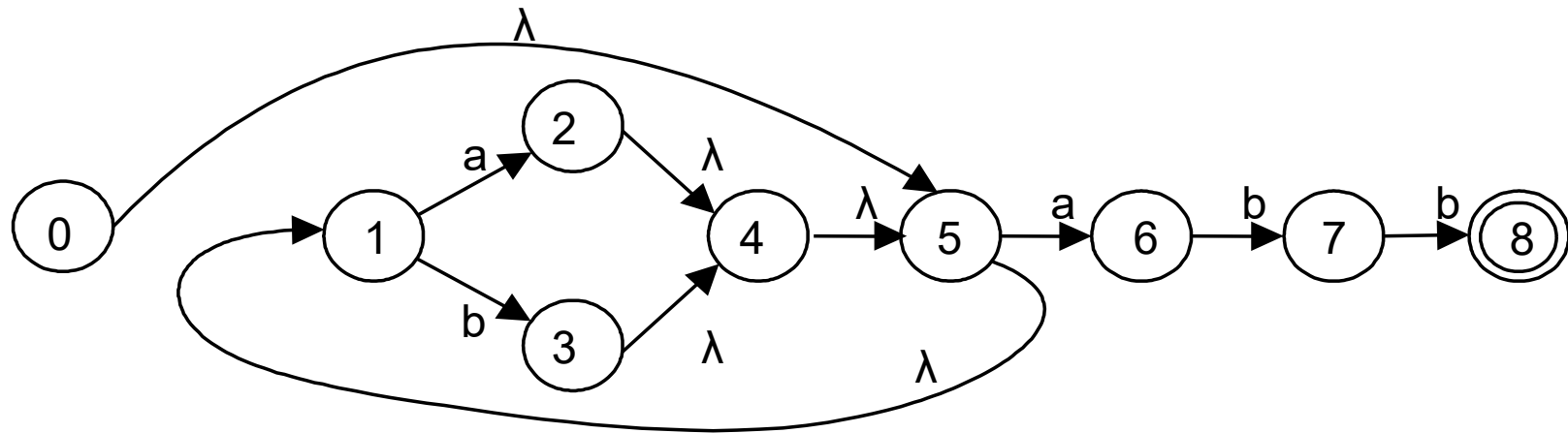
```
    construir estado  $U := \text{cierre-}\lambda(0)$ 
    añadir  $U$  a estadosD como estado no marcado
mientras haya un estado no marcado  $T$  en estadosD
hacer
    marcar  $T$ 
    para cada símbolo de entrada a hacer
         $U := \text{cierre-}\lambda(\text{mueve}(T, a))$ 
        si  $U$  no está en estadosD entonces
            añadir  $U$  a estadosD como
estado no marcado
         $\text{tranD}[T, a] := U$ 
    fin
fin
```



Autómatas Finitos Deterministas

Conversión de un AFN en un AFD

- Ejemplo AFN para $(a|b)^* abb$.
 - Estado de inicio del AFD es $A = \text{cierre-}\lambda(\{0\}) = \{0, 1, 5\}$

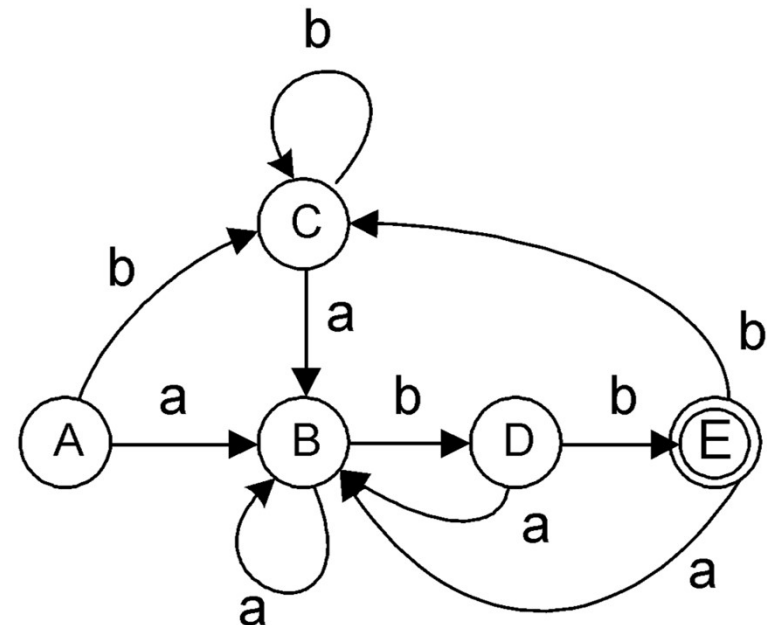


Autómatas Finitos Deterministas

Conversión de un AFN en un AFD

- Ejemplo AFN para $(a|b)^* abb$.
 - Estado de inicio del AFD es $A = \text{cierre-}\lambda(\{0\}) = \{0, 1, 5\}$

Estado	Símbolo de Entrada	
	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C



Autómatas Finitos Deterministas

Minimización de un AFD

- Minimización de estados de un AFD.
 - Todo conjunto regular es reconocido por un AFD con el mínimo número de estados que es único.
 - Algoritmo.
 1. Se construye una partición inicial P del conjunto de estados con dos grupos: Estados de aceptación F y estados de no-aceptación $S-F$.



Autómatas Finitos Deterministas

Minimización de un AFD

- Minimización de estados de un AFD.

- Algoritmo (cont.).

- 2. Aplicar el siguiente procedimiento para construir una nueva partición P_{nueva} .

- para** cada grupo G de P **hacer**

- crear partición de G en subgrupos que cumplan que dos estados

- s y t de G estén en el mismo subgrupo sii para todos los

- símbolos de entrada a , s y t tienen transiciones en a hacia estados del mismo grupo de P ;

- sustituir G en P_{nueva} por el conj. de todos los subgrupos formados;

- fin**



Autómatas Finitos Deterministas

Minimización de un AFD

- Minimización de estados de un AFD.
 - Algoritmo (cont.).

3. Si $P_{\text{nueva}} = P$, hacer $P_{\text{final}} := P$ y continuar con el paso 4. Si no, hacer $P := P_{\text{nueva}}$ y continuar con el paso 2.
4. Escoger en cada grupo de la partición un estado representante, que formará parte del AFD mínimo.

Construir la tabla de transiciones utilizando los estados representantes.

El estado inicial del AFD mínimo es el representante del grupo que contiene el estado s_0 del AFD inicial.

Los estados finales son los representantes que están en F .



Autómatas Finitos Deterministas

Minimización de un AFD

- Minimización de estados de un AFD.
 - Algoritmo (cont.).
- 5. Eliminar todos los estados inactivos (estados de no aceptación que tiene transiciones hacia él mismo con todos los símbolos de entrada)
Eliminar estados inalcanzables desde el estado inicial.
Todas las transiciones a estos estados quedan indefinidas.

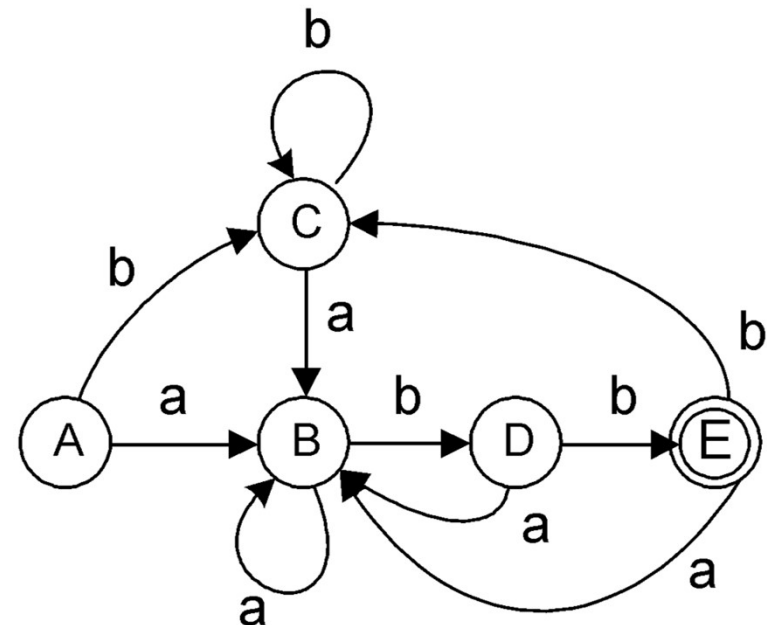


Autómatas Finitos Deterministas

Minimización de un AFD

- Ejemplo de minimización de AFD para $(a|b)^*abb$.
 - Inicialmente, $P = (ABCD) (E)$.

Estado	Símbolo de Entrada	
	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C

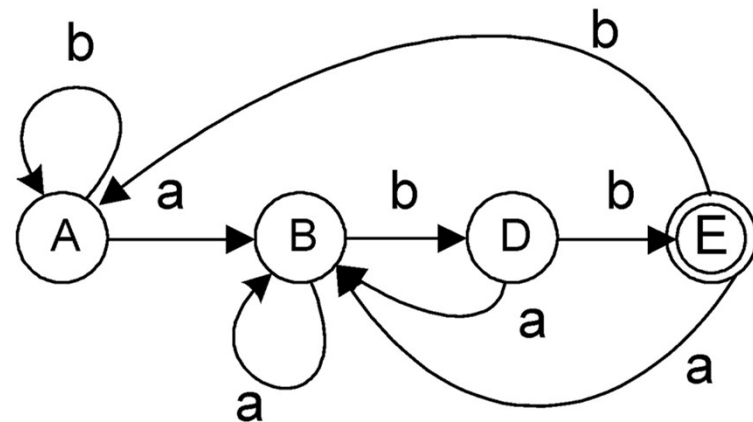


Autómatas Finitos Deterministas

Minimización de un AFD

- Ejemplo de minimización de AFD para $(a|b)^*abb$.
 - Inicialmente, $P = (ABCD) (E)$.

Estado	Símbolo de Entrada	
	a	b
A	B	A
B	B	D
D	B	E
E	B	A

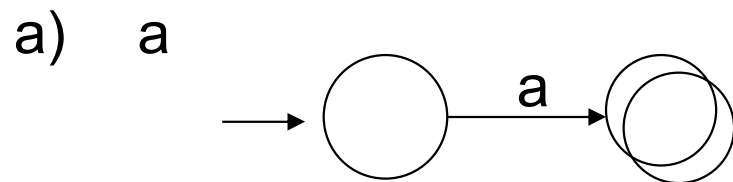
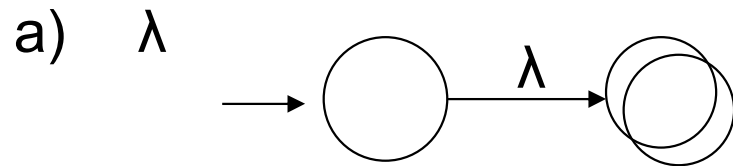
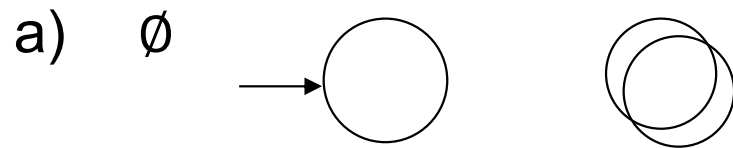


Teoremas de kleene

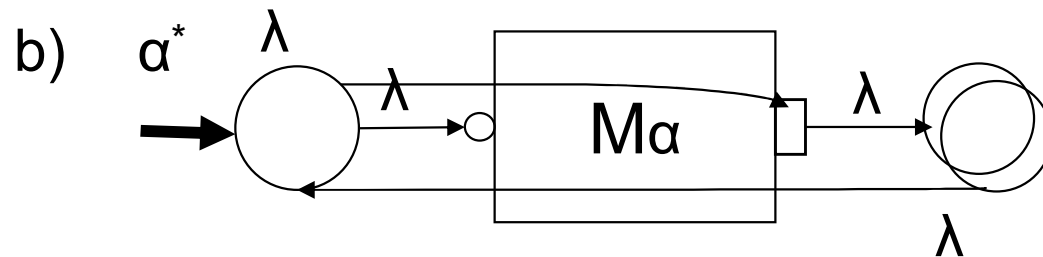
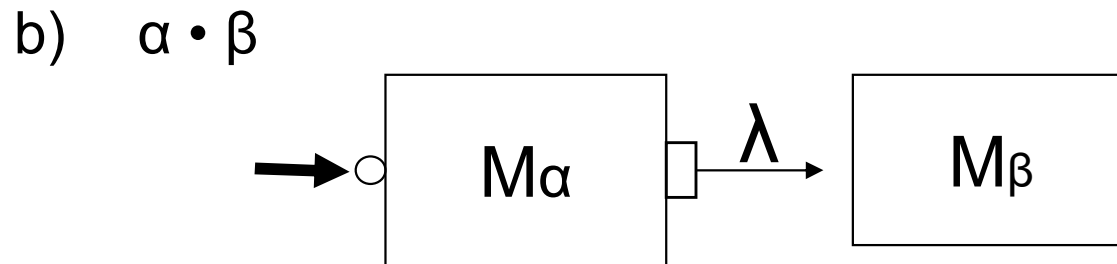
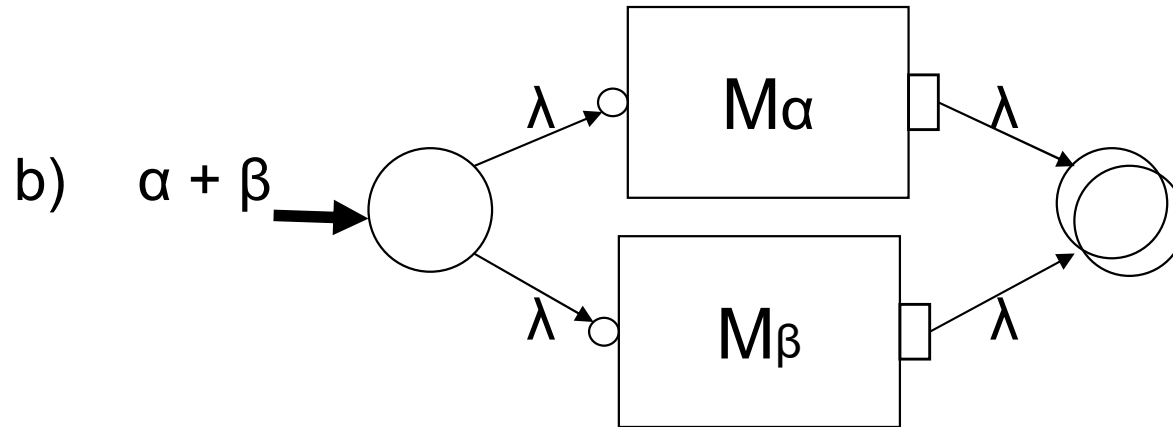


ER $\Rightarrow$ λ -AFND

- Buscamos autómatas para cualquier expresión regular:



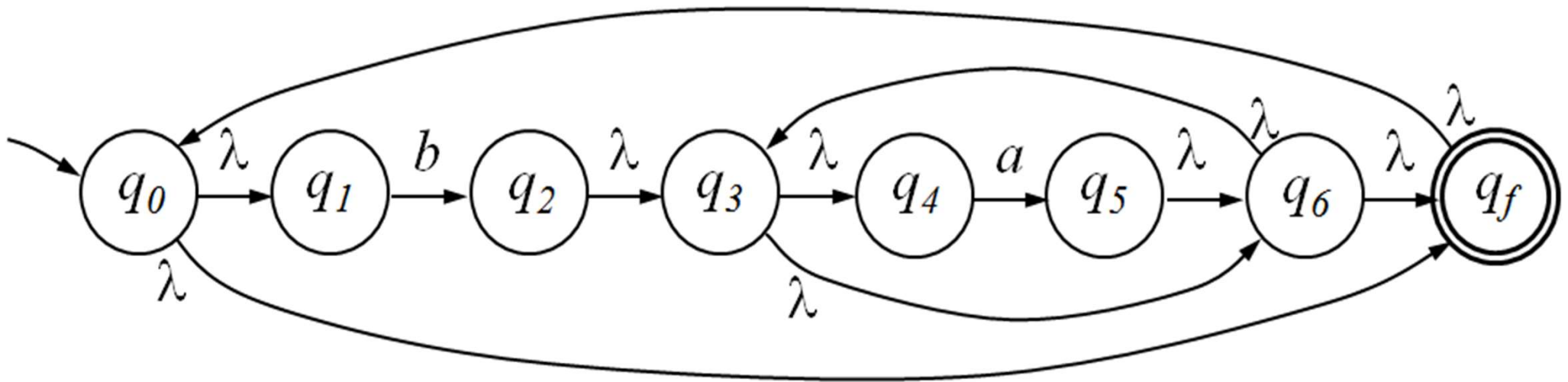
ER $\Rightarrow$ λ -AFND



Teorema de síntesis de Kleene

Ejemplo: La siguiente expresión regular es equivalente al siguiente AFND:

$$(ba^*)^*$$



Teorema de análisis de Kleene

- Recordatorio: Un lenguaje es regular si se cumple:

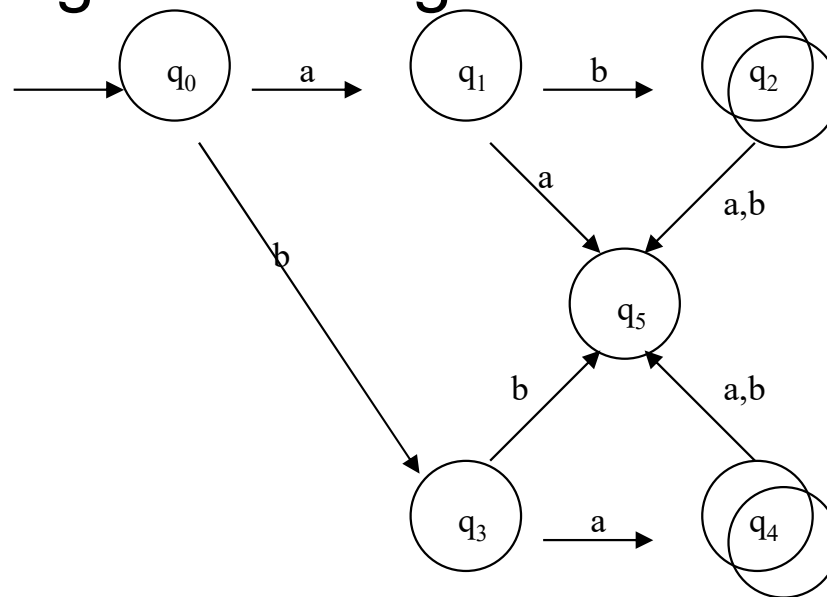
- 1- L es $\varnothing$
- 2- L es $\{\lambda\}$
- 3- L es $\{a\}$ con $a \in \Sigma$
- 4- L es $L_1 \cup L_2$, $L_1 \cdot L_2$, L_1^* con $L_1, L_2 \in L.R$

- Teorema 4.3: Teorema de análisis de Klenne: todo lenguaje aceptado por un AF es caracterizado por una ER: $AF \Rightarrow ER$
- Conclusión: Por los teoremas de síntesis y análisis de Kleene concluimos que el conjunto de los lenguajes regulares es el mismo que el conjunto de los lenguajes aceptados por un autómata finito.



T^a de análisis de Kleene

- Demostración T^a de análisis de Kleene
 - A la vez que definimos un algoritmo que demuestra el T^a, lo vamos aplicando sobre un ejemplo.
 - Supongamos el siguiente Autómata Finito:



Tª de análisis de Kleene

1) **Construimos conjuntos x_i** de la siguiente forma:

$$x_i = \{ w \in \Sigma^* \mid \Delta(q_i, w) \cap F \neq \emptyset \}$$

Cada x_i representa el conjunto de cadenas sobre Σ que hacen que M pase desde q_i hasta un estado final (conjunto de cadenas aceptadas por q_i).

Formamos un **sistema de ecuaciones**

• Ejemplo:

$$x_0 = a x_1 + b x_3$$

$$x_1 = b x_2 + a x_5$$

$$x_2 = \lambda + a x_5 + b x_5$$

$$x_3 = a x_4 + b x_5$$

$$x_4 = \lambda + a x_5 + b x_5$$

$$x_5 = \emptyset$$



Tª de análisis de Kleene

2) **Resolvemos** el sistema de ecuaciones por sustitución intentando obtener la expresión de x_0 y que será la expresión regular aceptada por el autómata finito.

- Ejemplo: Obtenemos:

$$x_0 = a b + b a$$

esta es la expresión regular solución que caracteriza el lenguaje regular aceptado por nuestro autómata finito.



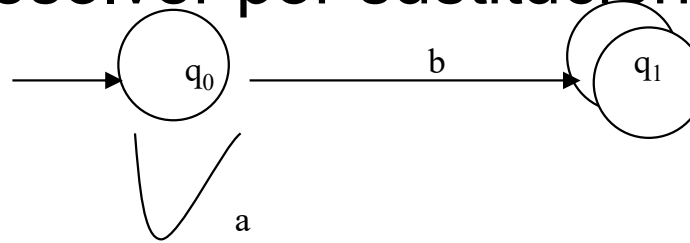
Lema de Arden

- En ocasiones aparecen expresiones recursivas que no se pueden resolver por sustitución.

– Ejemplo:

$$x_0 = a x_0 + b x_1$$

$$x_1 = \lambda$$



- Lema de Arden: $X = AX + B$, $\lambda \notin A \Rightarrow X = A^*B$

– Ejemplo:

Resolvemos aplicando Lema de Arden:

$$x_1 = \lambda \Rightarrow (\text{sustitución}) \Rightarrow$$

$$x_0 = a x_0 + b \Rightarrow (\text{Arden}) \Rightarrow$$

$$x_0 = a^*b$$

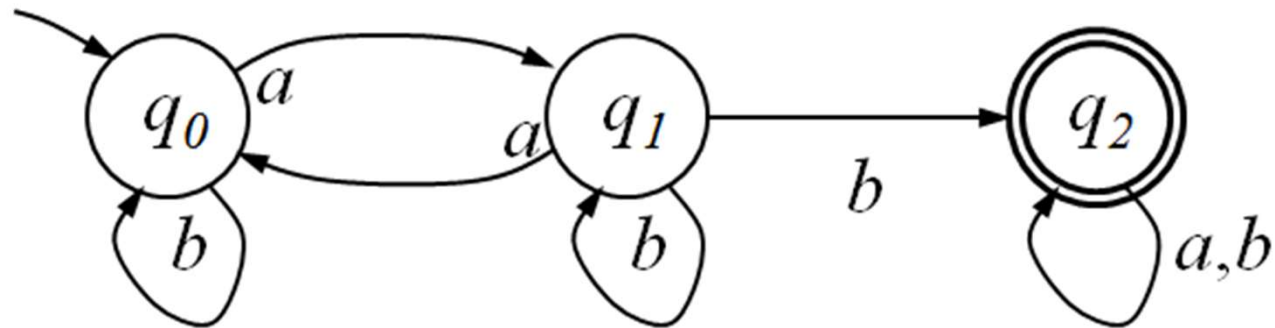
ER que caracteriza el AF de ejemplo:

$$\alpha = a^*b$$



T^a de análisis de Kleene

Ejemplo Teorema de análisis de Kleene



Las ecuaciones de los estados son:

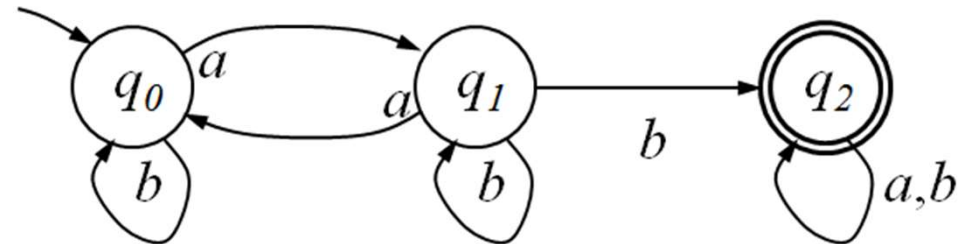
$$x_0 = bx_0 + ax_1$$

$$x_1 = ax_0 + bx_1 + bx_2$$

$$x_2 = \lambda + (a+b)x_2$$

Tª de análisis de Kleene

Ejemplo Teorema de análisis de Kleene



- Aplicando el teorema de Arden, se despeja x_2 de la tercera ecuación:

$$x_2 = (a+b)^*.\lambda = (a+b)^*$$

- Sustituimos este valor de x_2 en la segunda ecuación:

$$x_1 = ax_0 + bx_1 + b(a+b)^*$$

- Aplicando el teorema de Arden, despejamos x_1 :

$$x_1 = b^*(ax_0 + b(a+b)^*) = b^*ax_0 + b^*b(a+b)^*$$

- Sustituimos este valor de x_1 en la primera ecuación:

$$x_0 = bx_0 + ab^*ax_0 + ab^*b(a+b)^*$$

- Aplicando el teorema de Arden, despejamos x_0 :

$$x_0 = (b+ab^*a)^*ab^*b(a+b)^*$$



2.5. La clase de los lenguajes regulares

Sean L , L_1 y L_2 lenguajes regulares. Entonces:

- $L_1 \cup L_2$ es LR
- $L_1 L_2$ es LR
- L^* es LR
- L^c es LR
- $L_1 \setminus L_2$ es LR
- $L_1 \cap L_2$ es LR

Demostración: (ejercicio)



2.6. Problemas de decisión de los lenguajes regulares

- Dado L un lenguaje, debemos hacernos las siguientes preguntas (problemas de decisión):
 - ¿Es L vacío?
 - ¿Es L finito / infinito?
 - ¿Es L regular?
 - ¿ $w \in L$?



2.6. Problemas de decisión de los lenguajes regulares

- Lema de Bombeo: Sea L un LR infinito. Entonces existe una constante n de forma que, si w es una cadena de L cuya longitud es mayor o igual que n , se tiene que $w=uv^ix$, siendo $uv^ix \in L$ para todo $i \geq 0$, con $|v| \geq 1$ y $|uv| \leq n$
- Observación: El lema de Bombeo me sirve para demostrar que un lenguaje NO es un LR, pero si un lenguaje cumple el lema de bombeo no puedo asegurar que sea un LR.



Propiedades de los Lenguajes Regulares

- Sea M un AF con k estados
 1. $L(M)$ no es vacío si y sólo si acepta una cadena de longitud menor de k
 2. $L(M)$ es infinito si y sólo si M acepta una cadena de longitud n , donde $k \leq n \leq 2k$
- $L = \{a^n b^n / n \in \mathbb{N}\}$ **no** es un Lenguaje Regular

Demostración: (ejercicio)



Esquema general de la asignatura

